МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"



КУРСОВА РОБОТА
з дисципліни
“ПРИКЛАДНЕ ПРОГРАМУВАННЯ”
на тему: **Організації API та бази даних для Сервісного Центру**

Студент КН-215
спеціальності 122 “Комп’ютерні науки”
Лозовий Олег Андрійович
Керівник
д. т. н., Виклюк Я.І

Кількість балів: _____ Оцінка:_____
Члени комісії

_____
(підпис) (вчене звання, науковий ступінь, прізвище та ініціали)

_____
(підпис) (вчене звання, науковий ступінь, прізвище та ініціали)

_____
(підпис) (вчене звання, науковий ступінь, прізвище та ініціали)

Львів – 2022

Зміст:

1. Опис завдання
2. Етапи реалізації завдання
    a. Модель даних
    b. REST API
    c. Object Relation Mapping (ORM)
    d. Бізнес-логіка
    e. Тестування
3. Аналіз результатів та інструкція користувача
4. Висновок
5. Список використаних джерел
6. Додатки

Опис завдання

Завданням курсової роботи з прикладного програмування було розроблення серверної частини системи обліку Сервісного центру.

Реалізовано такі аспекти, як: облік пристроїв у сервісному центрі, слідкування за прогресом ремонту техніки зданої у сервісний центр, а також збереження інформації про самі ж сервіси.

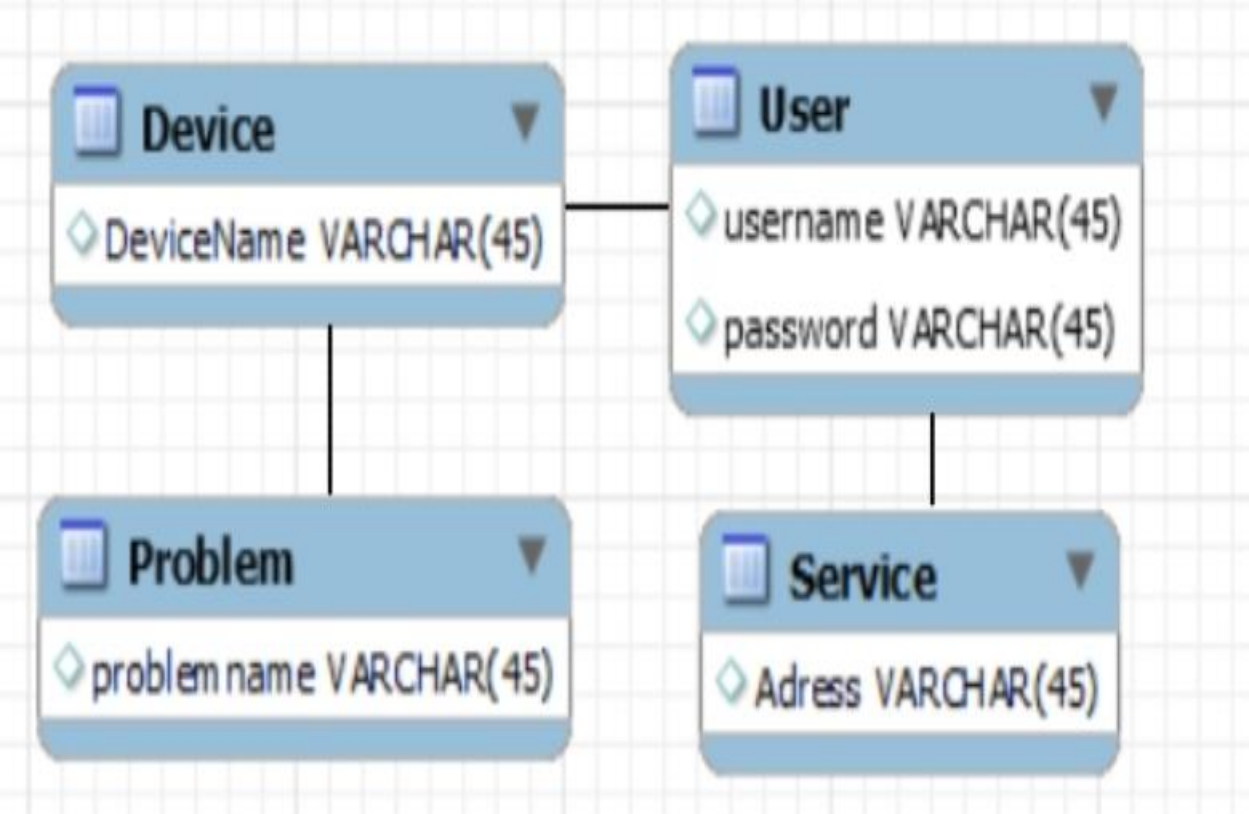
Курсова робота була написана за декілька етапів:

- налаштування сервера
- проектування REST API
- налаштування ORM
- реалізація API
- реалізація авторизації
- тестування.

Реалізація завдання.

а) Модель даних

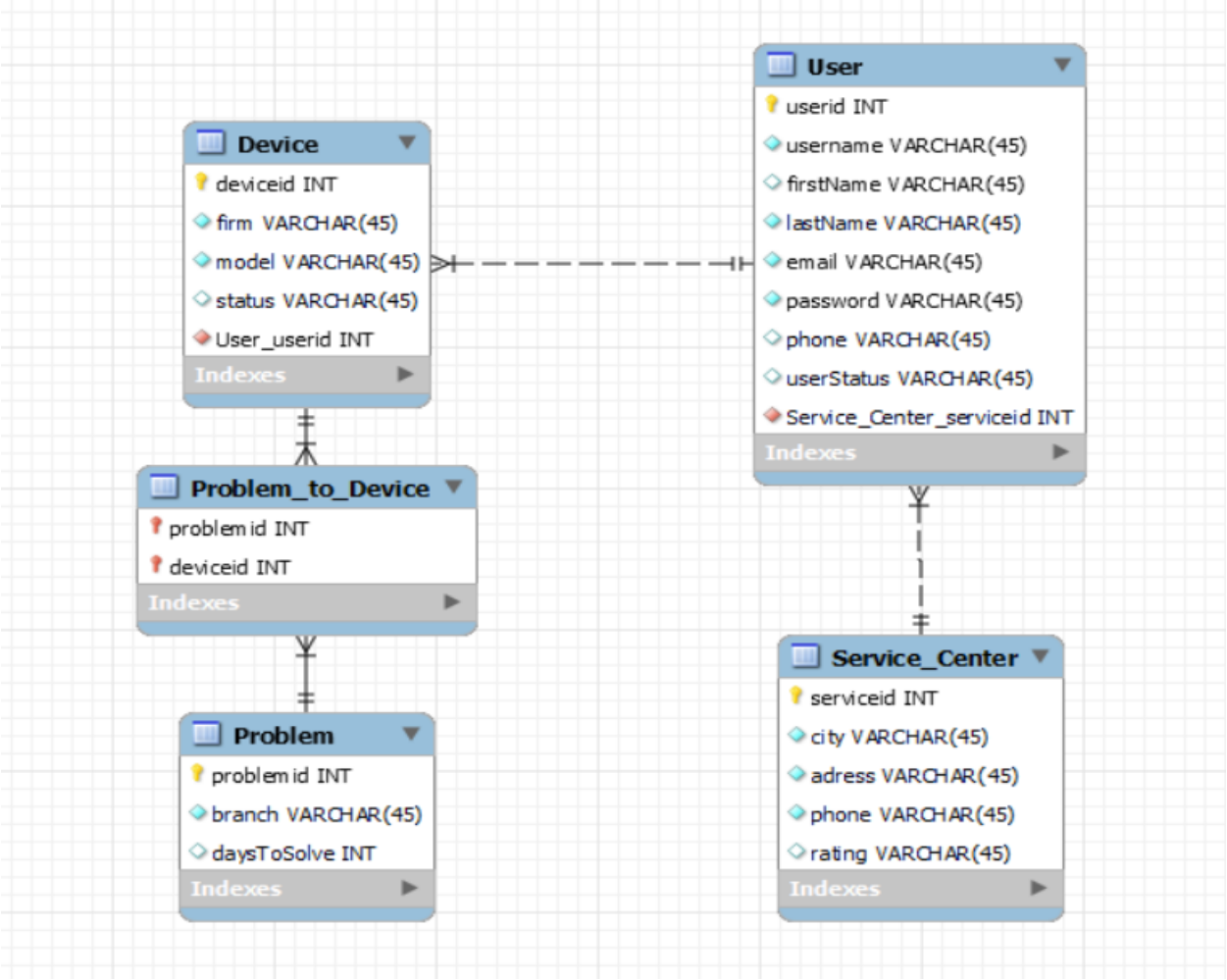
ER діаграма:



Загалом, є 4 головні сутності: сервіс, користувач, пристрій та проблема. Користувачі поділяються на 4 типи: адміністратор, майстер, власник пристрою та неаторизований користувач.

Майстер при отримані пристрою записує, що він його отримав. Як тільки він почав його ремонтувати, також ставить відповідний статус. По завершеню ремонту пристрій отримує статус “done”, ф отже готовий до повернення власнику. В цей ж час власник пристрою може відслідковувати прогрес ремонту.

Спроектована база даних:



У базі данних зберігаємо всю необхідну користувачу інформацію про сервіс, необхідну для майстра інформацію про пристрій цього користувача, а також тип проблеми, які містить даний пристрій. Також для кожного користувача зберігаємо інформацію про нього, та його username та password(у хешованому виді)

b) REST API

Для написання методів був використаний модуль *flask*.

Flask — це легка структура веб-додатків WSGI. Він розроблений, щоб зробити початок роботи швидким і легким, з можливістю масштабування до складних програм. Flask дає пропозиції, але не накладає жодних залежностей або компонування проекту. Розробник сам вибирає інструменти та бібліотеки, які він хоче використовувати. Спільнота пропонує багато розширень, які спрощують додавання нових функцій.

Є 4 головні сутності: Service_Center, User, Device, Problem.

1. User:

   Опис:

   Користувач системи. Може бути як Майстром так і Користувачем сервісу, або ж Адміністратором.

   Схеми:

   User: Поля: id, username, firstname, lastname, email, password, phone, userStatus, service_centerid.

   Методи:

   [POST] /login: авторизація користувача на сайті. Доступ мають всі.

   [GET] /user: отримання всіх користувачів. Доступ лише у адміністратора.

   [POST] /user: створення нового користувача (sign up). Доступ мають всі.

   [PUT] /user: оновлення даних про користувача. Доступ має сам користувач та адміністратор.

   [GET] /user/{username}: отримання інформації про користувача за username. Доступ має сам користувач та адміністратор.

   [DELETE] /user/{username}: видалення інформації про користувача за username. Доступ має сам користувач, майстер та адміністратор.

   [GET] /user/{username}/device: отримання інформації про пристрої користувача за username. Доступ має сам користувач, майстер та

адміністратор.

2. Service:

Опис:

Ця сутність створена для зберігання основної інформації про сервісні центри.

Схеми:

Service_Center: Поля: serviceid, city, adress, phone, rating

Методи:

[GET] /service: отримання списку сервісів, Доступ мають усі користувачі.

[POST] /service: створення сервісу. Доступ має лише адміністратор.

[PUT] /service: оновлення даних сервіс. Доступ має адміністратор або ж майстер з цього сервісу.

[GET] /service/{service_centerid}: отримання сервісу за id. Доступ мають усі користувачі.

[DELETE] /service/{service_centerid}: видалення сервісу за id. Доступ має лише адміністратор.

[GET] /service/device/: отримання інформації про девайси в сервісах, посортовану за статусом. Доступ лише адміністратор.

3. Device:

Опис:

Містить інформацію про пристрій, його статус та власника.

Схеми:

Device: Поля: deviceid, firm, model, status, userid

Методи:

[GET] /device: отримання списку пристроїв. Доступ лише адміністратор.

[POST] /device: додавання пристрою. Доступ має адміністратор та майстер.

[PUT] /device: оновлення інформації про пристрій. Доступ має адміністратор та майстер.

[GET] /device/findByStatus: отримання списку пристроїв з вказаним статусом. Доступ має лише адміністратор.

[GET] /device/findByFirm: отримання списку пристроїв з вказаною фірмою. Доступ має лише адміністратор.

[GET] /device/{deviceid}: отримання пристрою за id. Доступ має лише майстер, адміністратор або власник пристрою.

[DELETE] /device/{deviceid}: видалення пристрою за id. Доступ має лише майстер, або адміністратор.

[GET] /device/{deviceid}/problem: отримання проблем пристрою за id. Доступ має лише майстер, адміністратор або власник пристрою.

[POST] /device/{deviceid}/problem: додавання проблем пристрою за id. Доступ має лише майстер або адміністратор.

Для реалізації звязку багато до багатьох створена таблиця: Problem_to_device з полями : deviceid, problemid.

[GET] /device/{deviceid}/problem/{problemid}: отримання проблеми пристрою за id пристрою та id проблеми. Доступ має адміністратор або власник пристрою.

[DELETE] /device/{deviceid}/problem/{problemid}: видалення проблеми пристрою за id пристрою та id проблеми. Доступ має адміністратор або майстер.

[PUT] /device/{deviceid}/problem/{problemid}: оновлення проблеми пристрою за id пристрою та id проблеми. Доступ має адміністратор або власник пристрою.

4. Problem:

Опис:

Містить інформацію про види проблем.

Схеми:

Problem: Поля: problemid, branch, daysToSolve

[GET] /problem: отримання списку проблем. Доступ у всіх авторизованих користувачів.

[POST] /problem: додавання проблеми. Доступ має адміністратор та майстер.

[PUT] /problem: оновлення інформації про проблему. Доступ має лише адміністратор.

[GET] /problem/{problemid}: отримання проблеми за id. Доступ у всіх авторизованих користувачів.

[DELETE] /problem/{problemid}: видалення проблеми за id. Доступ має лише адміністратор.

[GET] /problem/findByBranch: отримання інформації про проблему за її назвою категорії. Доступ у всіх авторизованих користувачів.

c) Object Relation Mapping (ORM)

Для кожної таблиці спроектованої бази даних, за допомогою sqlalchemy, створена ORM модель.

SQLAlchemy — інструментарій SQL та об'єктно-реляційне відображення для мови програмування Python випущене під ліцензією MIT. Він надає «повний набір добре відомих шаблонів корпоративного рівня стабільності, сконструйованих для високопродуктивного доступу до бази даних, написаних простою мовою Python».

Після створення моделей, була створена перша міграція за допомогою alembic(*інструмент для міграції бази даних*).

Alembic надає такі функції:

- *Може видавати оператори ALTER до бази даних, щоб змінити структуру таблиць та інших конструкцій*
- *Надає систему, за допомогою якої можна створювати «сценарії міграції»; кожен сценарій вказує на певну серію кроків, які можуть «оновити» цільову базу даних до нової версії, і, за бажанням, серію кроків, які можуть «понизити» аналогічно, виконуючи ті самі кроки у зворотному порядку.*
- *Дозволяє сценаріям виконуватися певним чином.*

Відповідно, коли будуть якісь зміни в моделях, можна легко створити нову міграцію і працювати далі.

d) Бізнес-логіка

Кожен метод має викликатись авторизованим користувачем(є винятки)

При створенні користувача, його пароль зберігається в базу в захешованому вигляді. Для перевірки паролю, також використовується цей модуль.

Для створення сервісу необхідно бути адміністратором сервісу.

Для додавання пристрою у список “на ремонт” необхідно бути адміністратором або майстром.

Для того щоб відслідковувати прогрес ремонту необхідно бути власником пристрою, майстром, або ж адміністратором.

Кожен користувач може переглядати список сервісів.

Для додавання(лінкування) видалення або оновлення проблем пристрою необхідно бути майстром або адміністратором.

Для отримання повної інформації про роботу сервісу, пристрої які у ньому знаходяться, проблем які вирішують у сервісі а також користувачів сайту, а також маніпуляцій з цією інформацією необхідно бути адміністратором.

Для перегляду списку проблем які вирішують у сервісі необхідно бути авторизованим користувачем.

Кожен користувач може змінювати дані лише свого акаунту.

Усі можливі помилки ловляться, у відповідь надсилаються ErrorMessages.

е) Тестування

Для тестування моєї програми було використане *pytest* тестування.

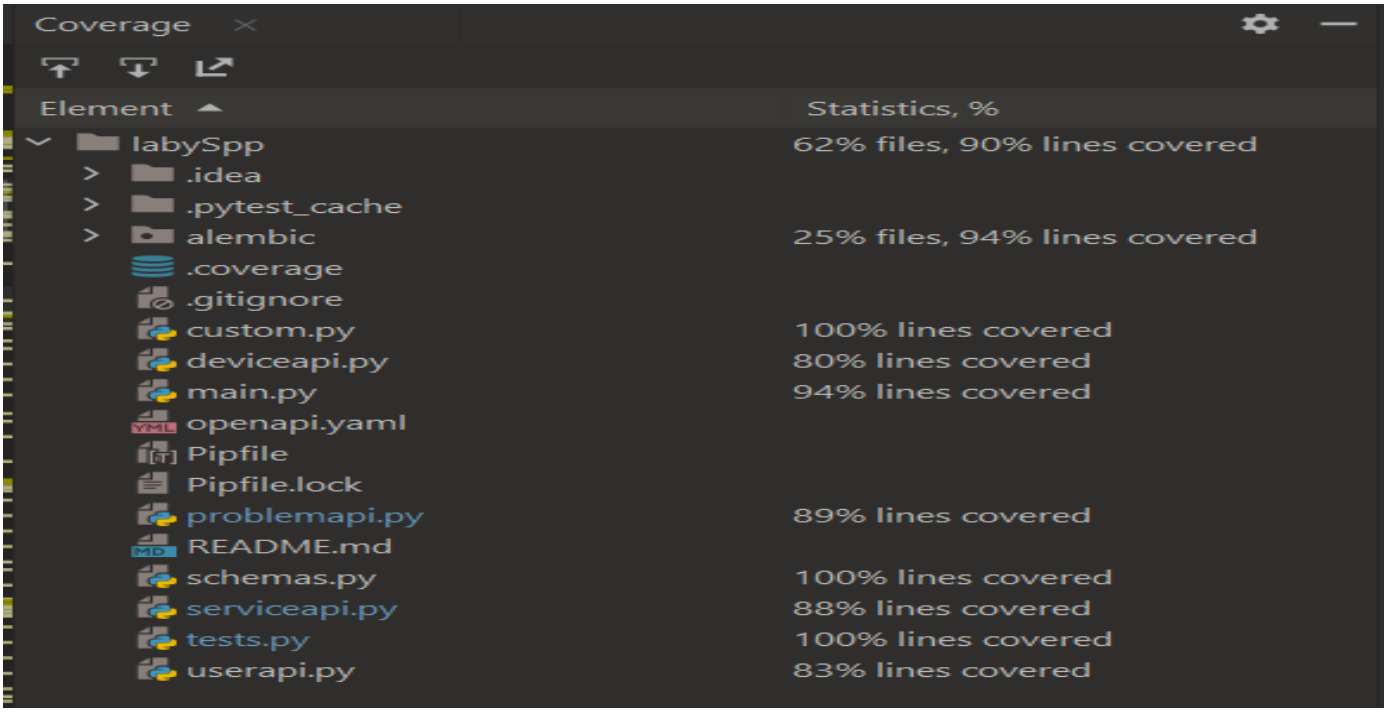
Pytest — це платформа тестування Python , яка походить від проекту PyPy .

Для цього були прописані тести, які перевіряли роботу всіх методів, ORM моделей, схем тощо.

За допомогою *coverage* було перевірено, яку частину програми перевірив *pytest*.

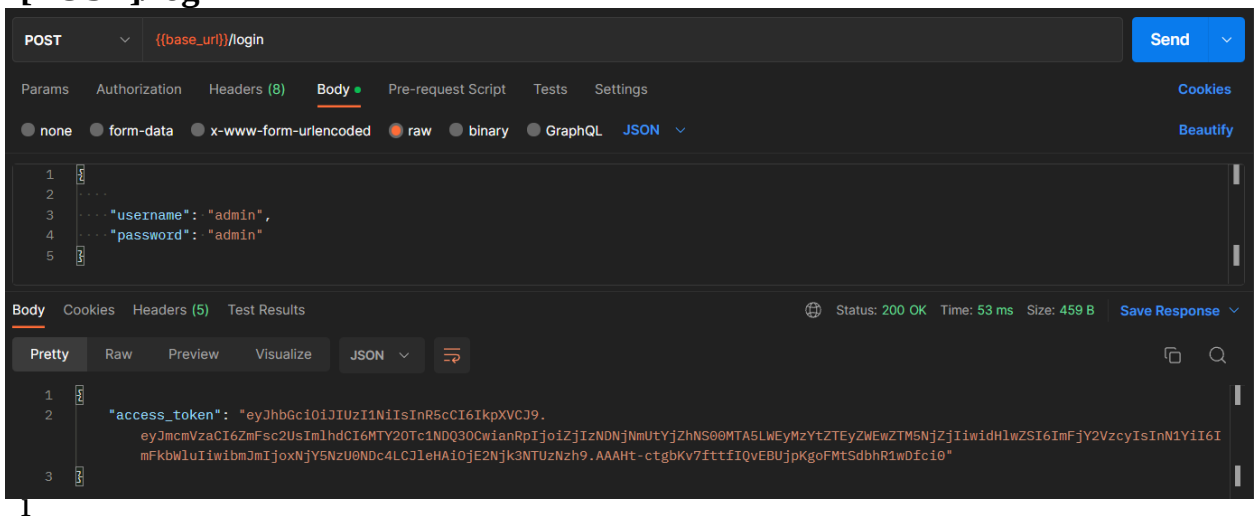
Coverage.py — це інструмент для вимірювання покриття коду програм Python. Він відстежує вашу програму, відзначаючи, які частини коду було виконано, а потім аналізує джерело, щоб визначити код, який міг бути виконаний, але не був виконаний. Вимірювання охоплення зазвичай використовується для вимірювання ефективності тестів. Він може показати, які частини вашого коду перевіряються тестами, а які ні.

Результати coverage:



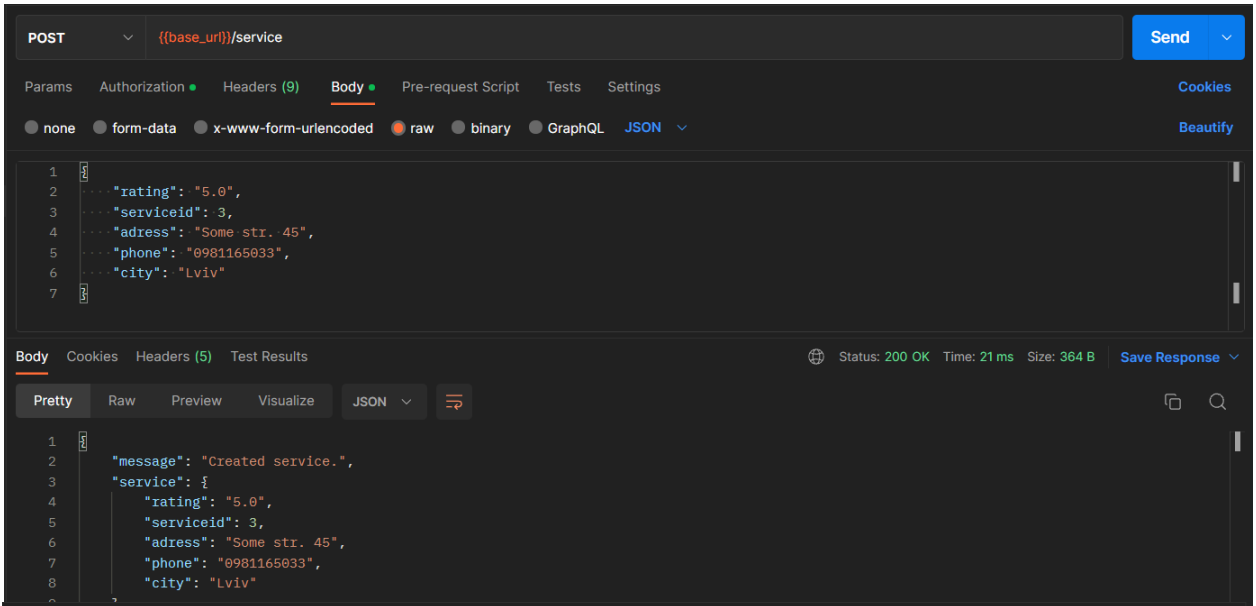
Аналіз результатів та інструкція користувача

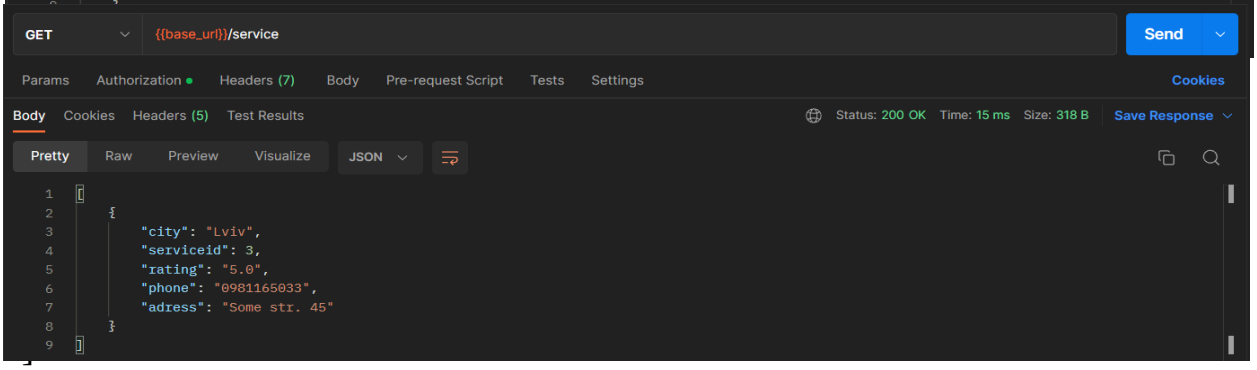
[POST]/login



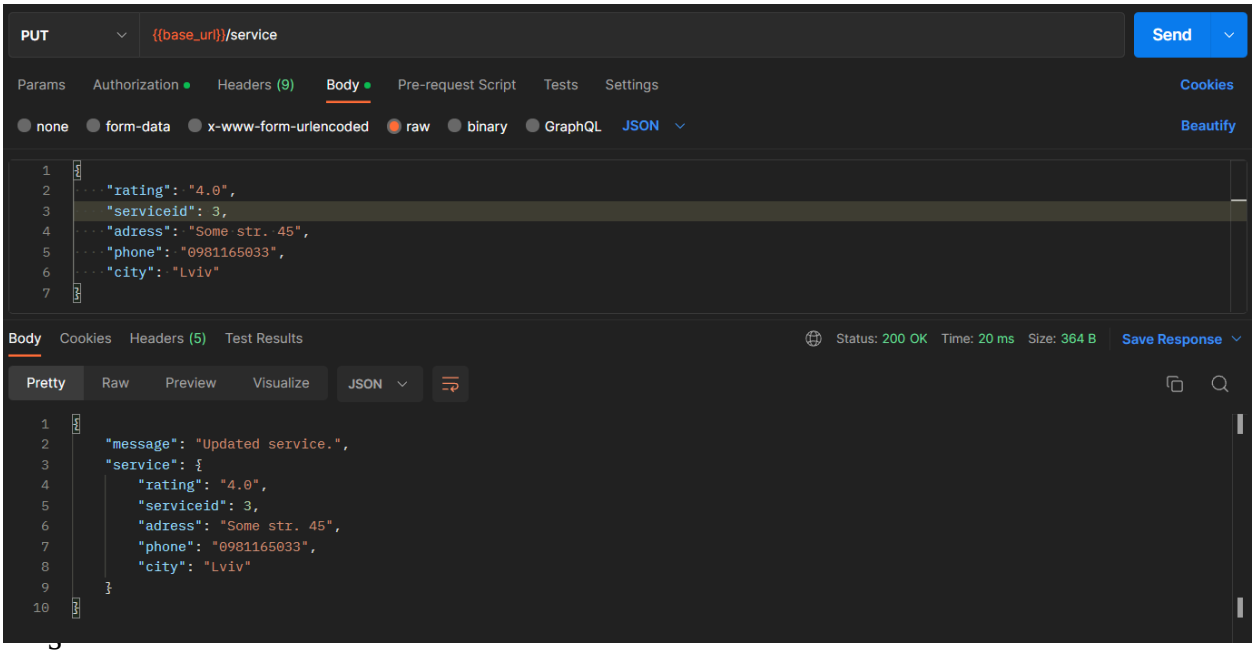
l

ce

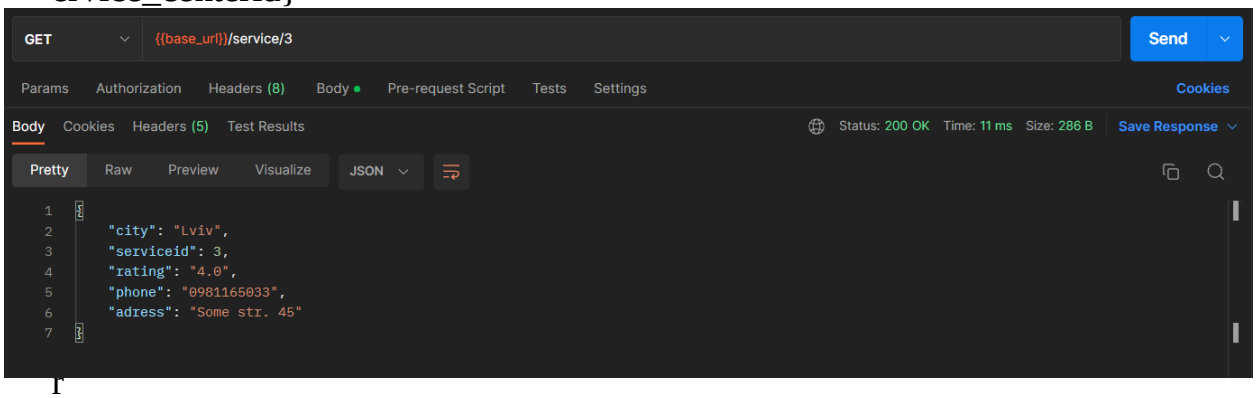




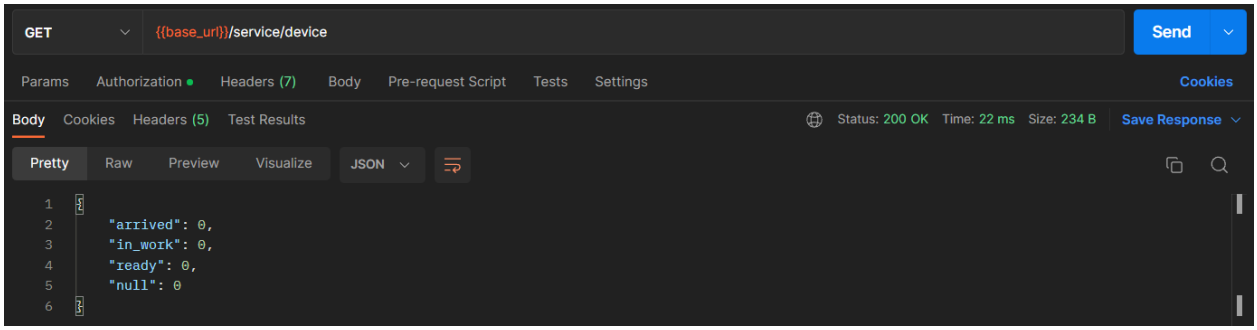
/service



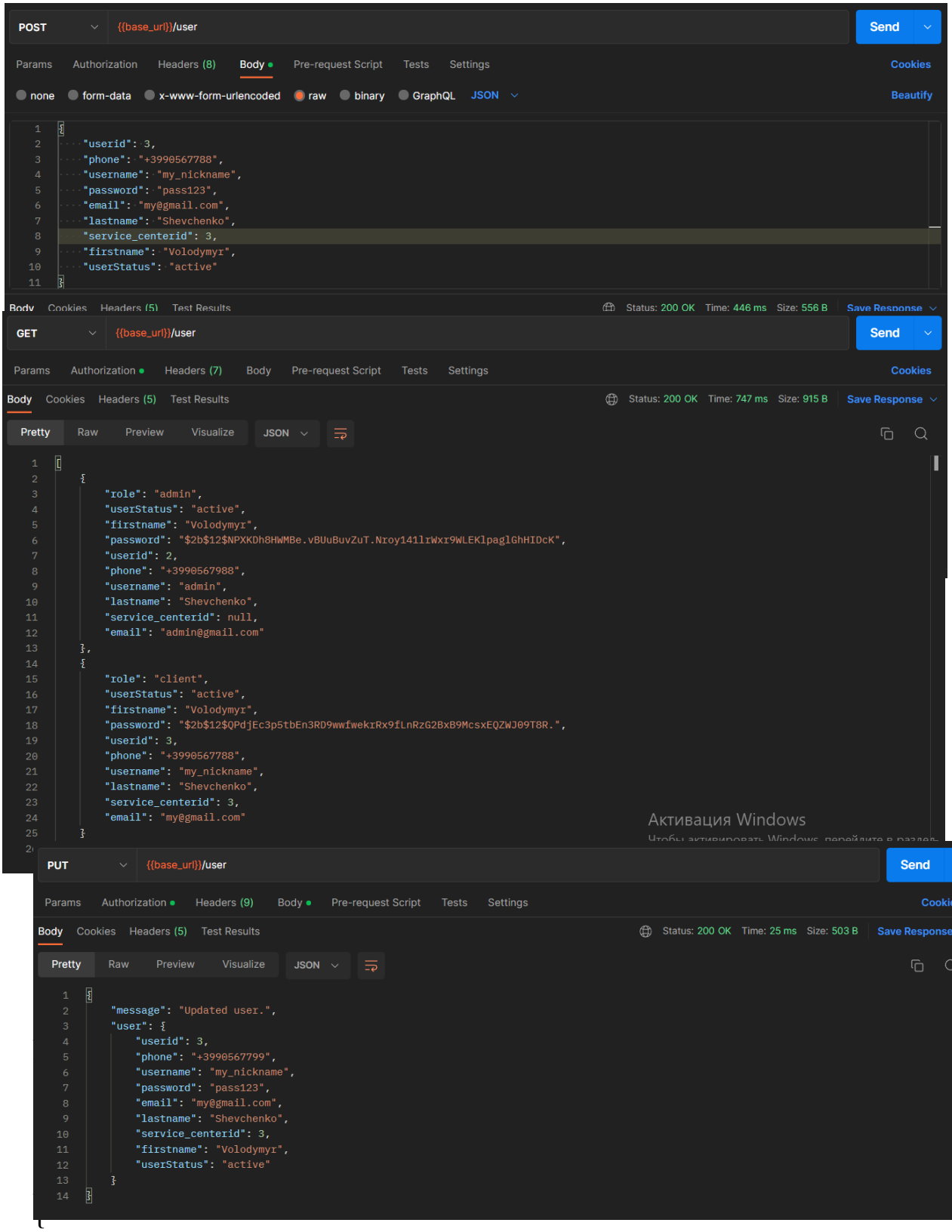
ervice_centerid}



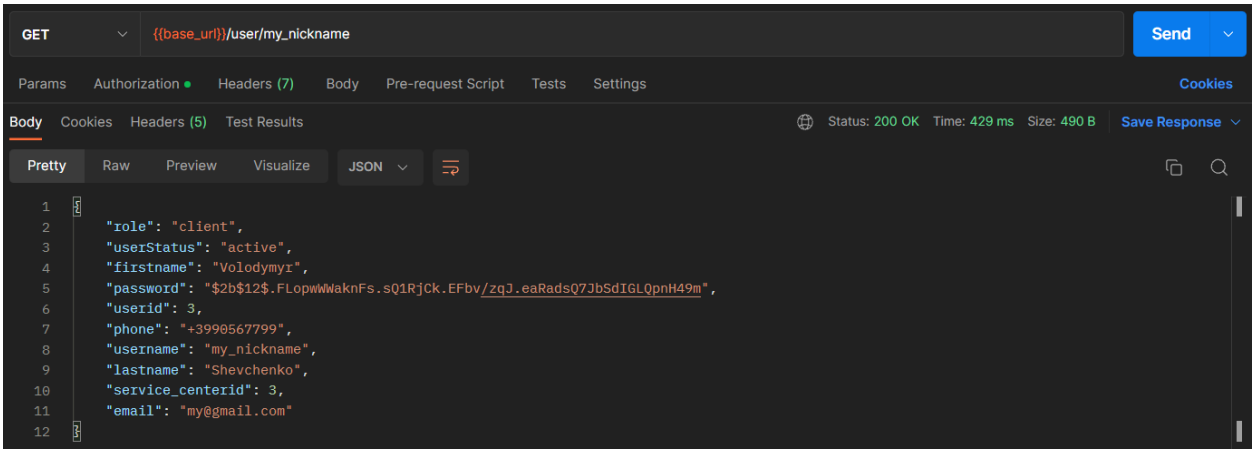
vice/device



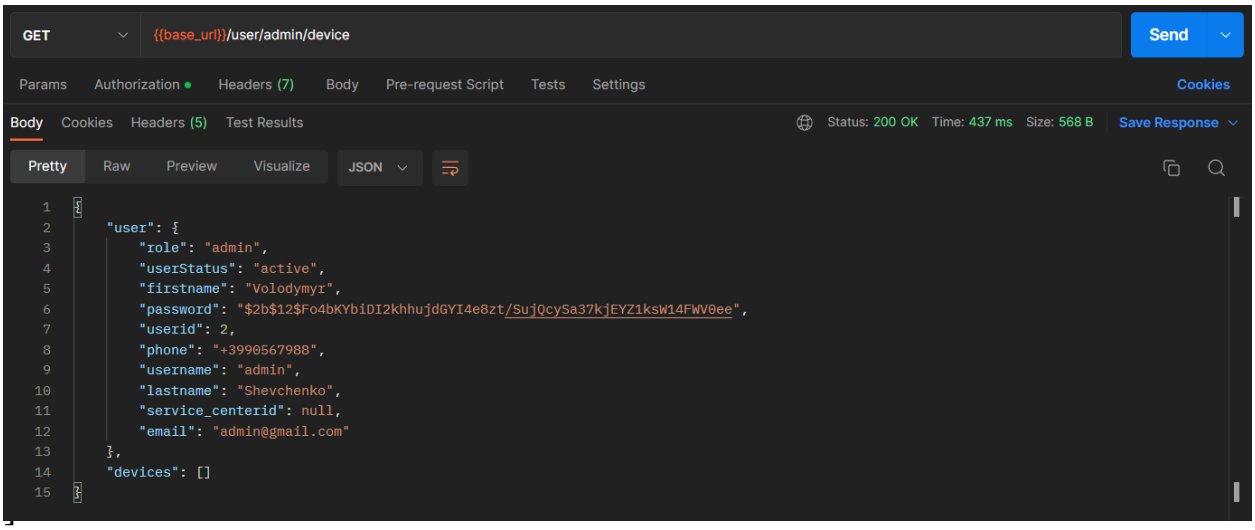
[POST]/user


POST
{{base_url}}/user
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"userid": 3,
"phone": "+3990567788",
"username": "my_nickname",
"password": "pass123",
"email": "my@gmail.com",
"lastname": "Shevchenko",
"service_centerid": 3,
"firstname": "Volodymyr",
"userStatus": "active"
}
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 446 ms
Size: 556 B
Save Response
GET
{{base_url}}/user
Send
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 747 ms
Size: 915 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
[
{
"role": "admin",
"userStatus": "active",
"firstname": "Volodymyr",
"password": "$2b$12$NPXKDh8HWMBe.vBUuBuvZuT.Nroy141lrWxr9WLEKlpaglGhHIDcK",
"userid": 2,
"phone": "+3990567988",
"username": "admin",
"lastname": "Shevchenko",
"service_centerid": null,
"email": "admin@gmail.com"
},
{
"role": "client",
"userStatus": "active",
"firstname": "Volodymyr",
"password": "$2b$12$QPdjEc3p5tbEn3RD9wwfwekrRx9fLnRzG2BxB9McsxEQZWJ09T8R.",
"userid": 3,
"phone": "+3990567788",
"username": "my_nickname",
"lastname": "Shevchenko",
"service_centerid": 3,
"email": "my@gmail.com"
}
Активация Windows
PUT
{{base_url}}/user
Send
Params
Authorization
Headers (9)
Body
Pre-request Script
Tests
Settings
Cookies
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 25 ms
Size: 503 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"message": "Updated user.",
"user": {
"userid": 3,
"phone": "+3990567799",
"username": "my_nickname",
"password": "pass123",
"email": "my@gmail.com",
"lastname": "Shevchenko",
"service_centerid": 3,
"firstname": "Volodymyr",
"userStatus": "active"
}
}

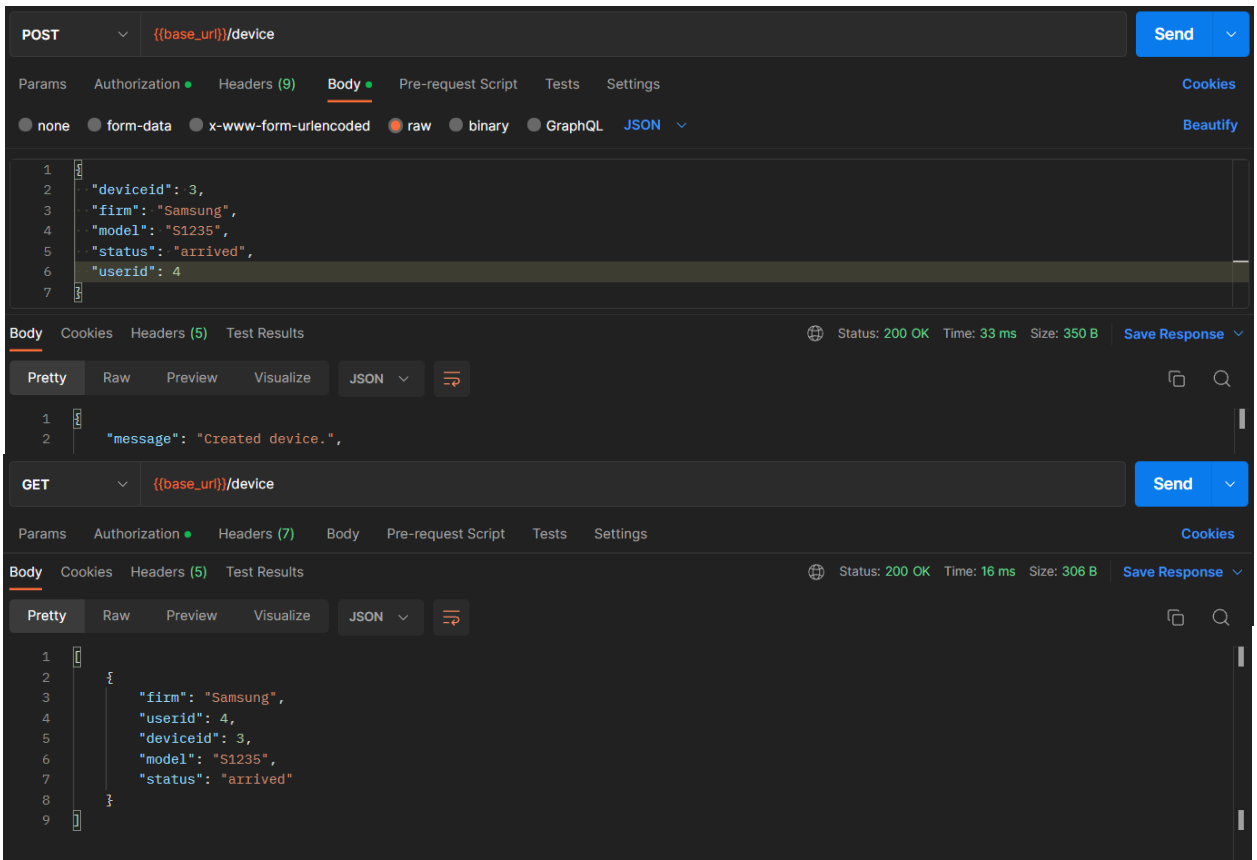
{
username}



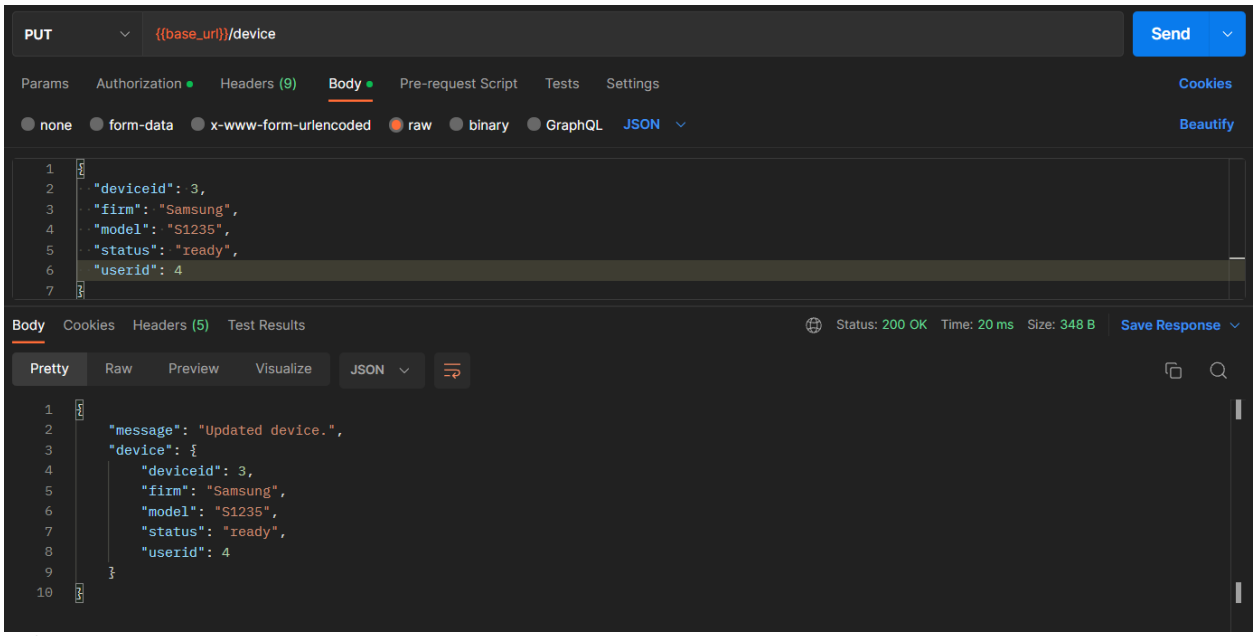
{username}/device



/device

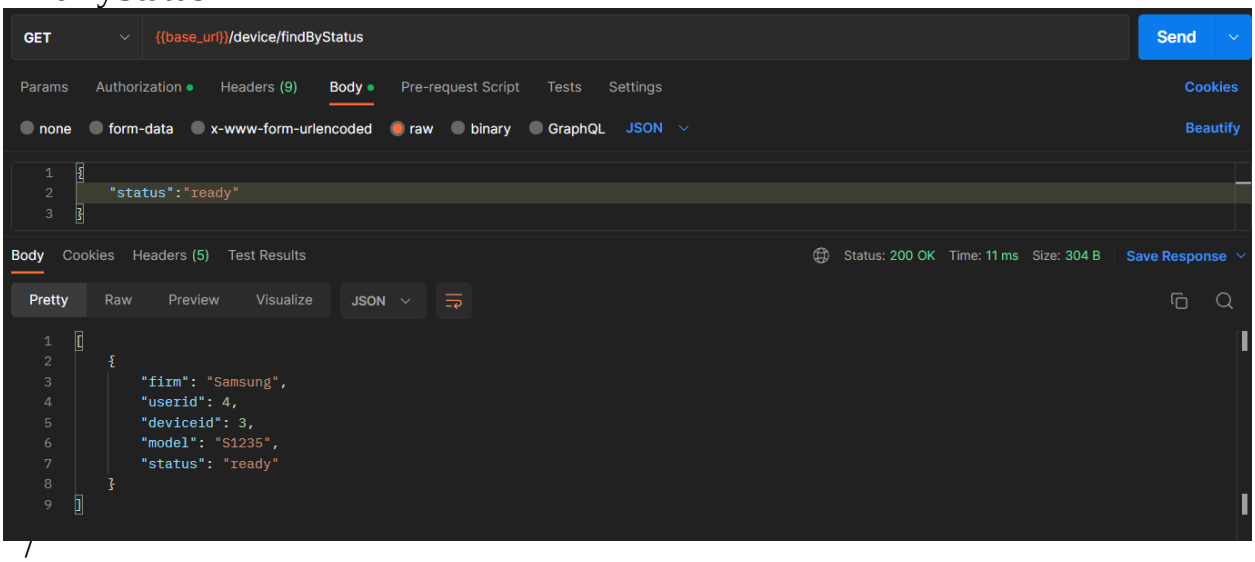


evice



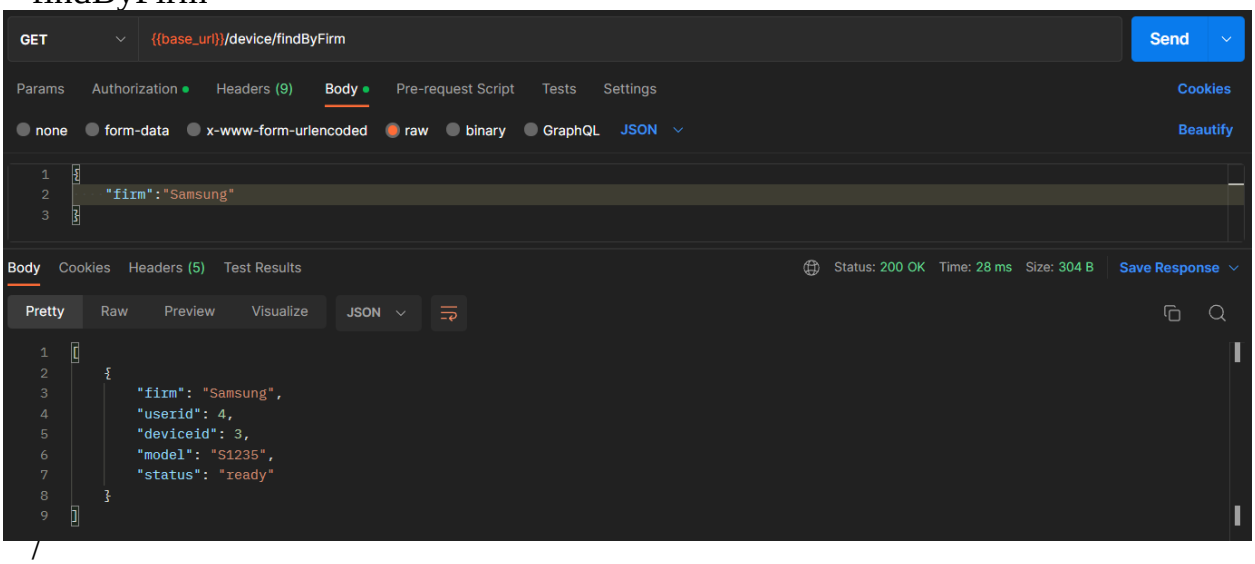
f

indByStatus



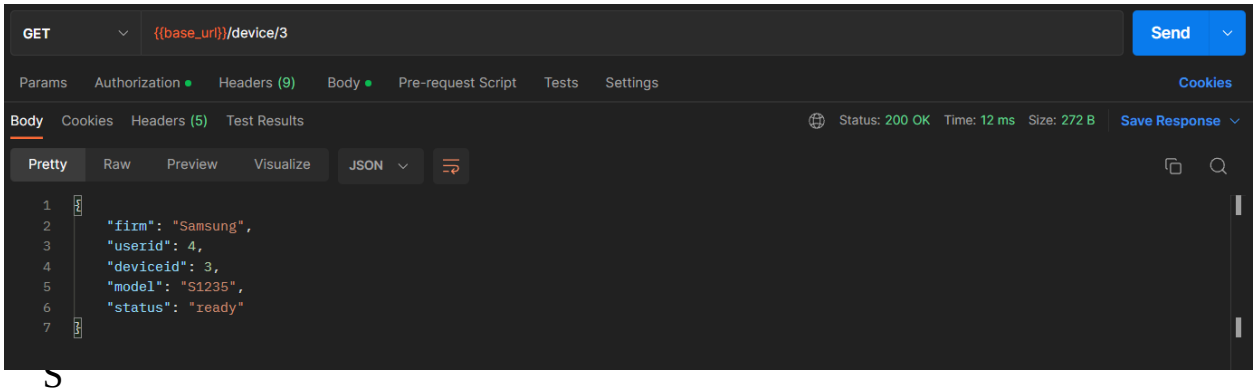
/

findByFirm

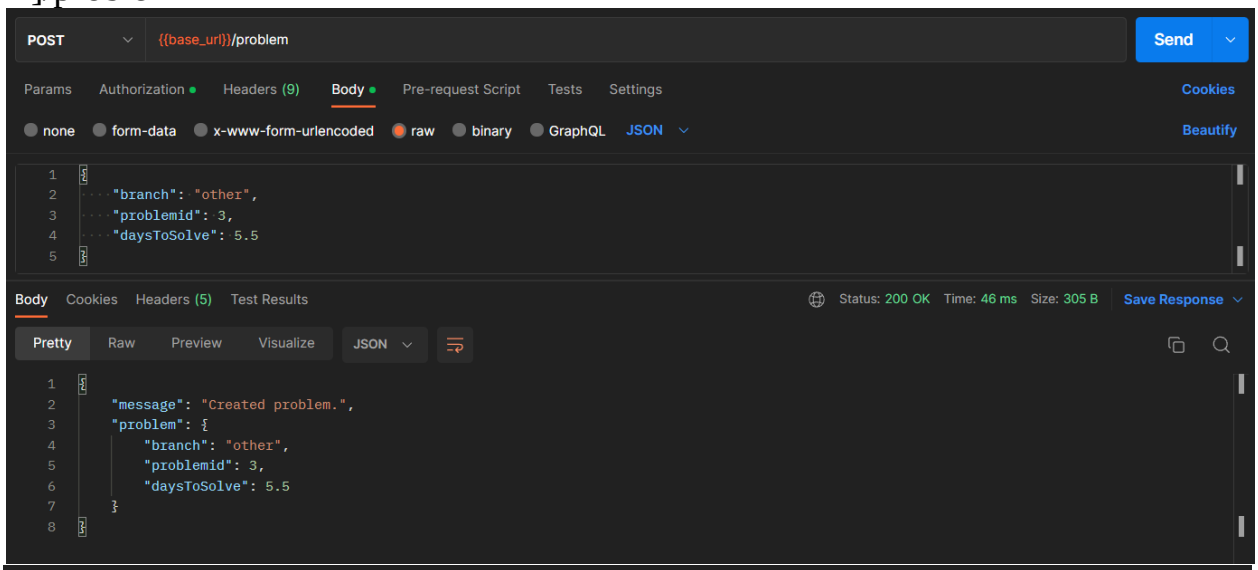


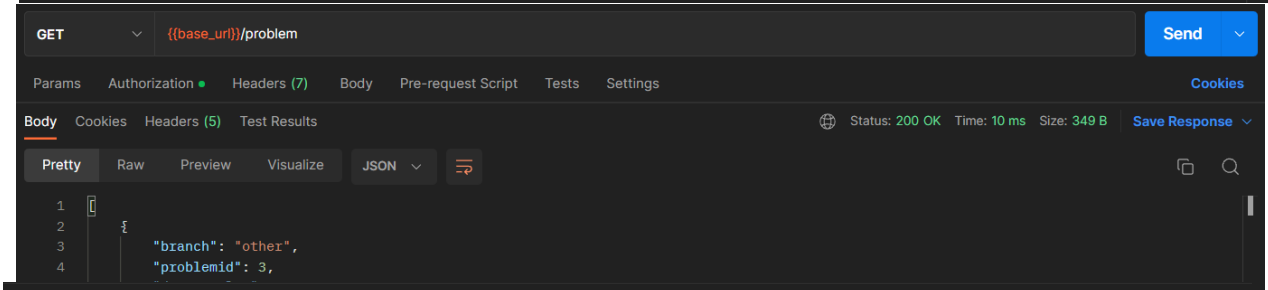
/

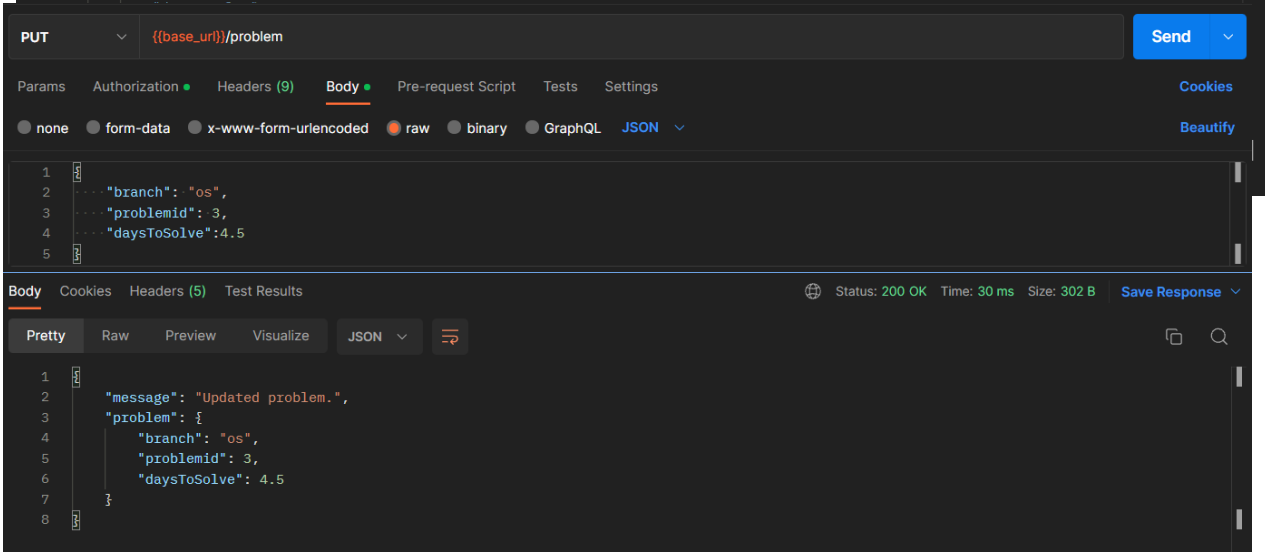
{deviceid}



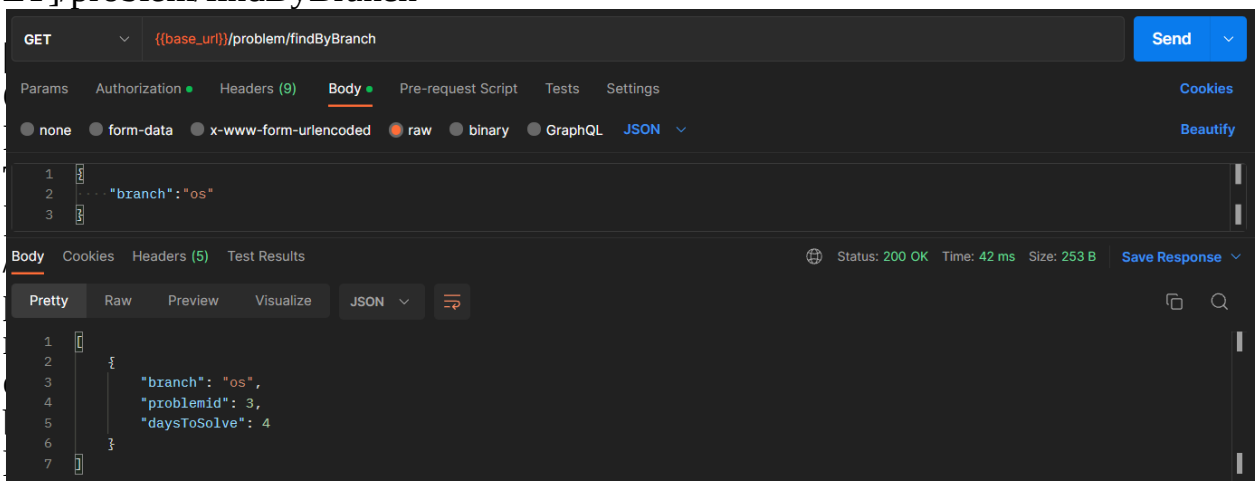
S
T]/problem



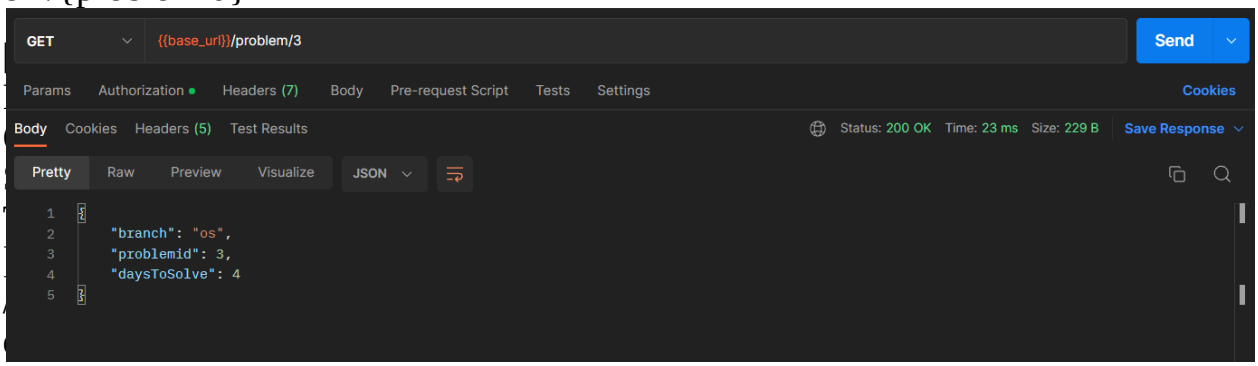




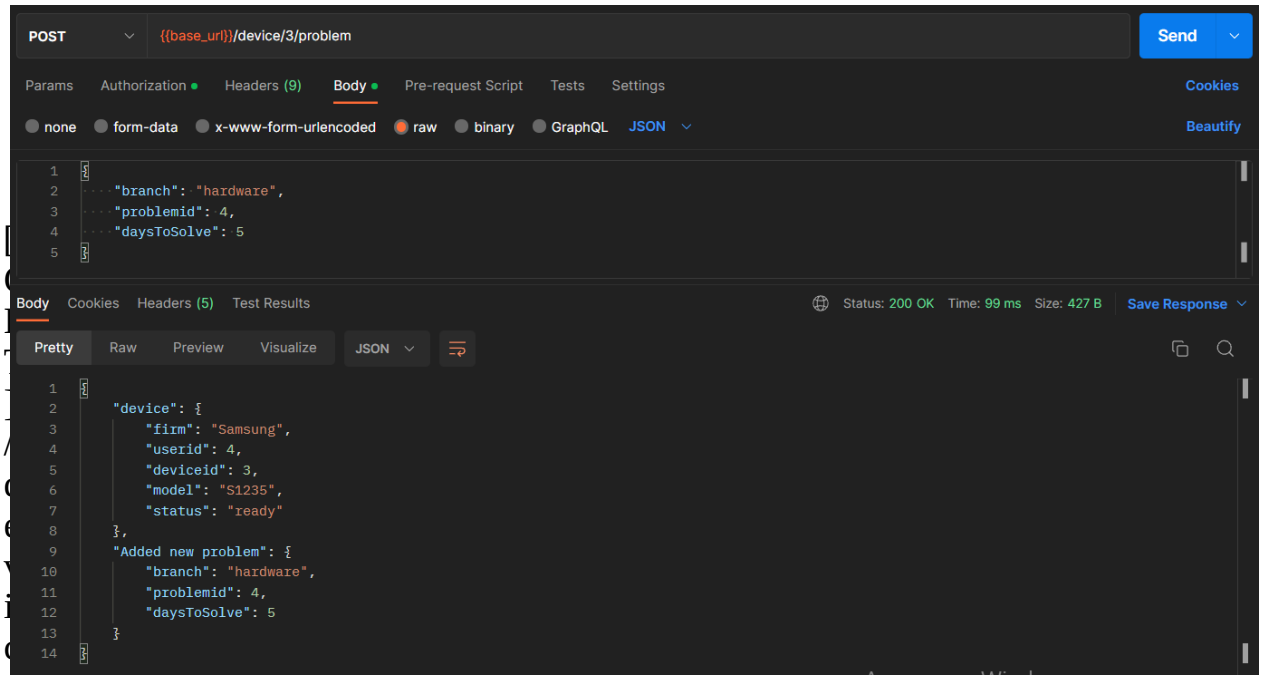
ET]/problem/findByBranch



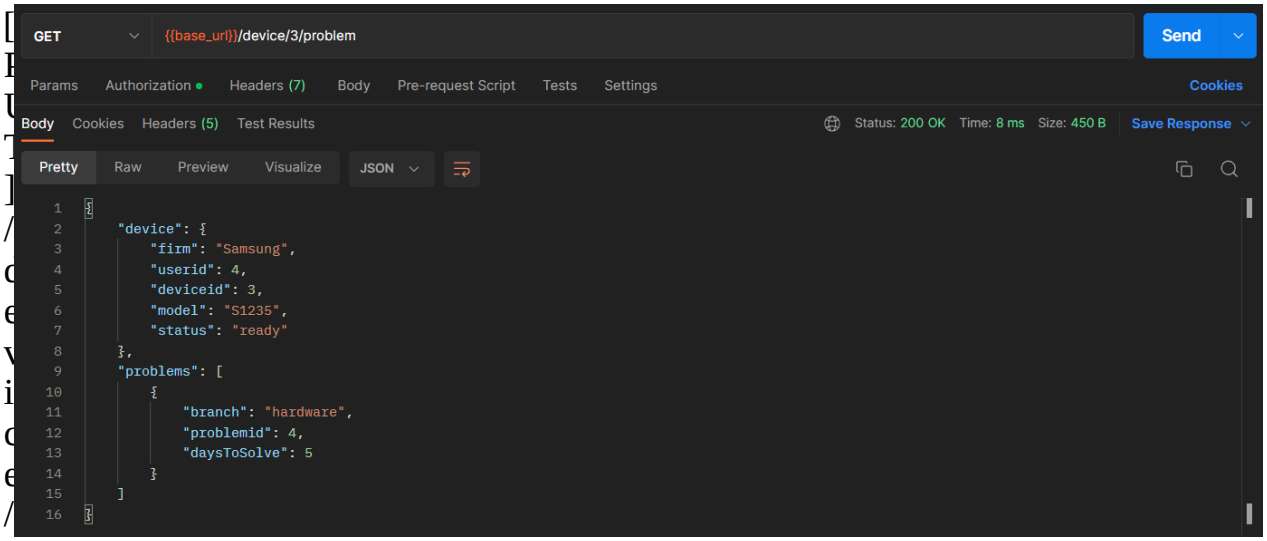
em/{problemid}



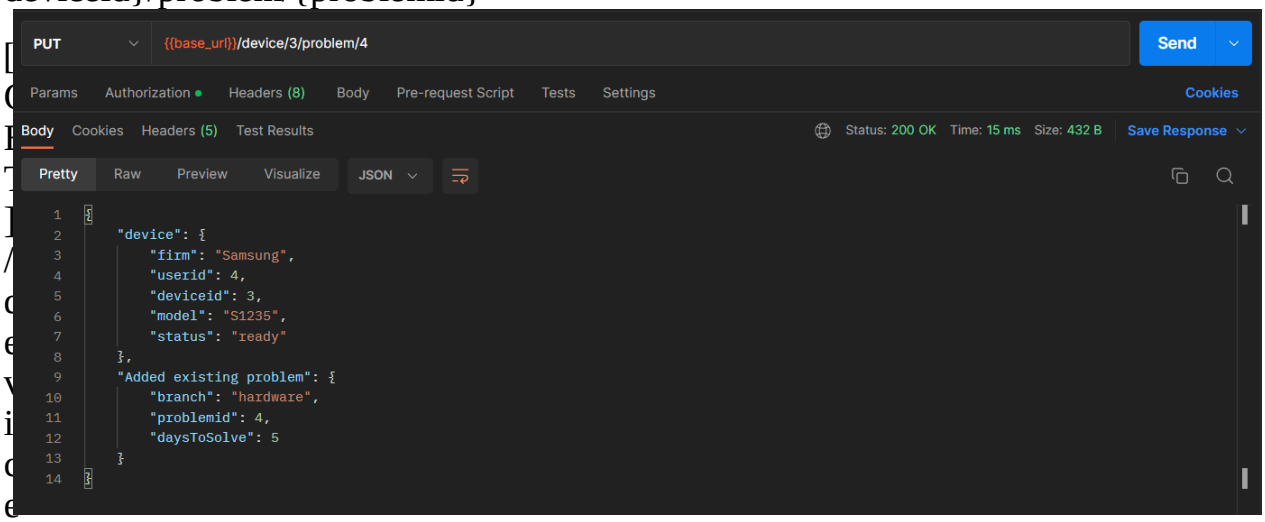
e
vice/{deviceid}/problem



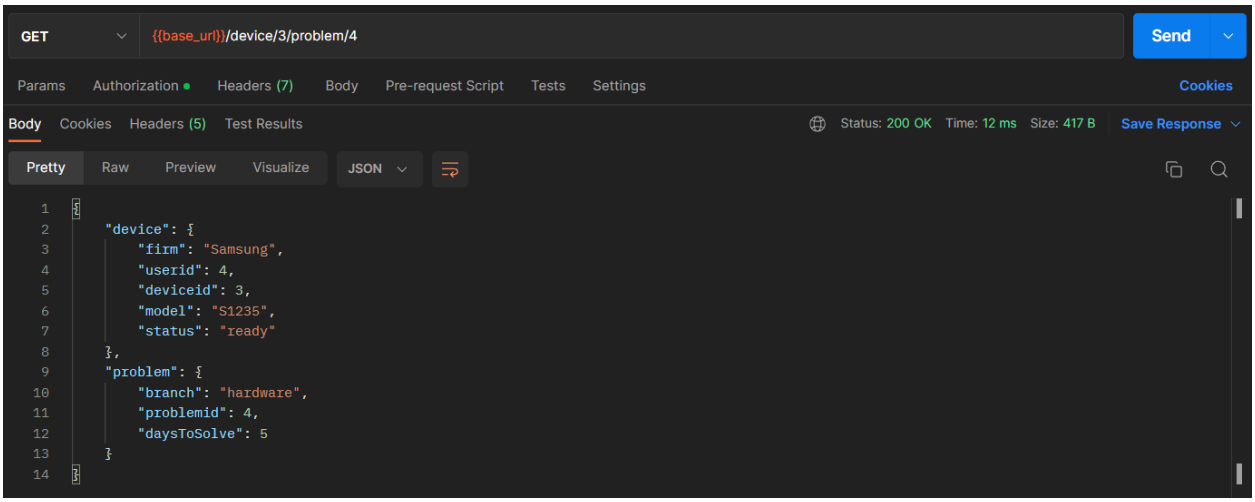
e
/{deviceid}/problem



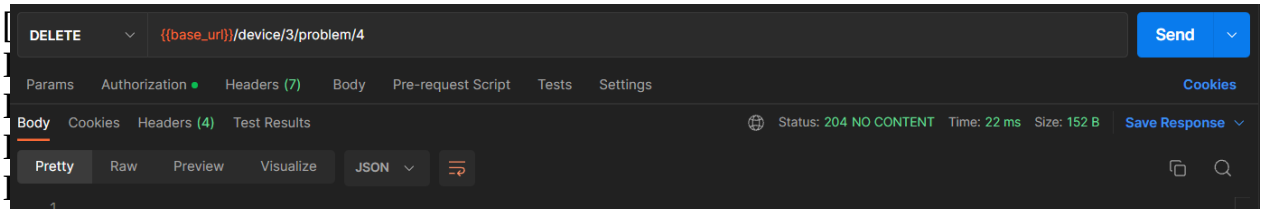
{deviceid}/problem/{problemid}



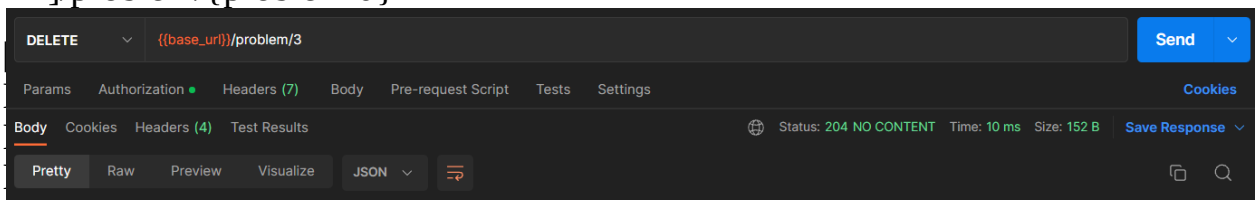
/{deviceid}/problem/{problemid}



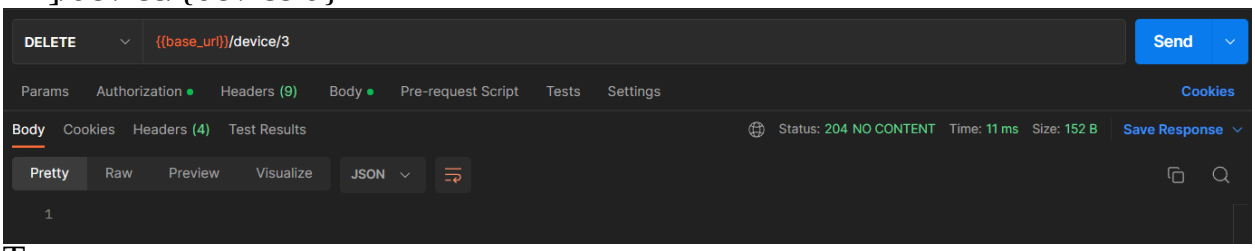
device/{deviceid}/problem/{problemid}



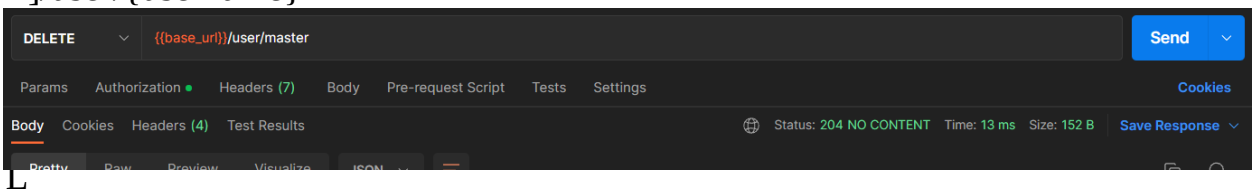
TE]/problem/{problemid}



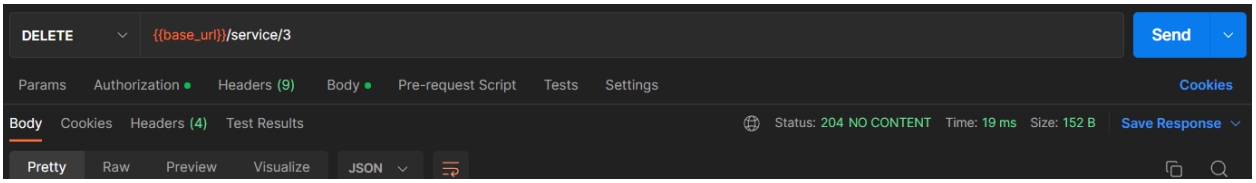
E
TE]/device/{deviceid}



T
E]/user/{username}



L
ETE]/service/{service_centerid}

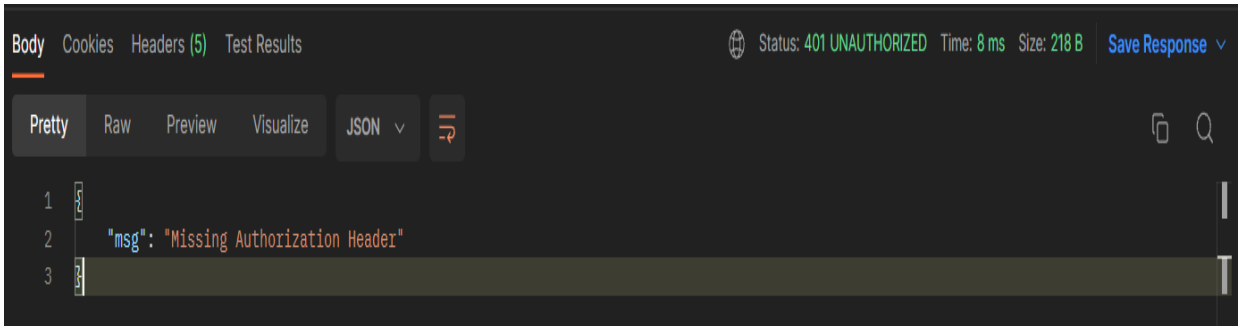


и
клади обробки помилок:

Неіснуючий об’єкт, 404



Неавторизований користувач



Висновок

Результатом цієї курсової роботи є готова серверна частина для сайту Сервісного центру. Для цього було спроектовано REST API та база даних, створено ORM моделі, додана автентифікація та авторизація. Логічним завершенням було тестування серверної частини.

Список використаних джерел

Flask: https://flask.palletsprojects.com/en/2.2.x/

SQLAlchemy: https://docs.sqlalchemy.org/en/14/index.html

Alembic: https://alembic.sqlalchemy.org/en/latest/

Marshmallow:https://marshmallow.readthedocs.io/en/stable/

Bearer Authentication: https://stackoverflow.com/questions/29931671/making-an-api-call-in-python-with-an-api-that-requires-a-bearer-token

Pytest: https://docs.pytest.org/en/7.2.x/

Coverage:https://coverage.readthedocs.io/en/stable/

Додатки

/alembic/migrations/versions:

0bd36dbb9974_create_model.py:

```
"""Create model

Revision ID: 0bd36dbb9974
Revises: 4e81a1f09f3c
Create Date: 2022-11-26 18:46:57.529651
```

```
    """
    from alembic import op
    import sqlalchemy as sa

    # revision identifiers, used by Alembic.
    revision = '0bd36dbb9974'
    down_revision = '4e81a1f09f3c'

    branch_labels = None
    depends_on = None


def upgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    pass
    # ### end Alembic commands ###

def downgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    pass
    # ### end Alembic commands ###
```

4e81a1f09f3c_fixed.py

```
"""Fixed

Revision ID: 4e81a1f09f3c
Revises: 8f820caf7bcd
Create Date: 2022-11-16 19:04:54.014809

"""
from alembic import op
import sqlalchemy as sa


# revision identifiers, used by Alembic.
revision = '4e81a1f09f3c'
down_revision = '8f820caf7bcd'
branch_labels = None
depends_on = None


def upgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    op.add_column('user', sa.Column('role', sa.String(length=32), nullable=False))
    op.drop_column('user', 'roles')
    # ### end Alembic commands ###


def downgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
```

```
    op.add_column('user', sa.Column('roles', sa.VARCHAR(length=32), autoincrement=False, nullable=False))
    op.drop_column('user', 'role')
    # ### end Alembic commands ###
```

8f820caf7bcd_create_tables.py

```
"""Create tables

Revision ID: 8f820caf7bcd
Revises:
Create Date: 2022-11-16 18:29:47.728516

"""
from alembic import op
import sqlalchemy as sa


# revision identifiers, used by Alembic.
revision = '8f820caf7bcd'
down_revision = None
branch_labels = None
depends_on = None


def upgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    op.create_table('problem',
    sa.Column('problemid', sa.Integer(), nullable=False),
    sa.Column('branch', sa.Enum('screen', 'battery', 'button', 'os', 'hardware', 'other', name='branchenum'), nullable=False),
    sa.Column('daysToSolve', sa.Integer(), nullable=True),
    sa.PrimaryKeyConstraint('problemid')
    )
    op.create_table('service_center',
    sa.Column('serviceid', sa.Integer(), nullable=False),
    sa.Column('city', sa.String(length=45), nullable=False),
    sa.Column('adress', sa.String(length=45), nullable=False),
    sa.Column('phone', sa.String(length=45), nullable=False),
    sa.Column('rating', sa.String(length=45), nullable=True),
    sa.PrimaryKeyConstraint('serviceid')
    )
    op.create_table('user',
    sa.Column('userid', sa.Integer(), nullable=False),
    sa.Column('username', sa.String(length=45), nullable=False),
    sa.Column('firstname', sa.String(length=45), nullable=True),
    sa.Column('lastname', sa.String(length=45), nullable=False),
    sa.Column('email', sa.String(length=45), nullable=False),
    sa.Column('password', sa.String(length=45), nullable=False),
    sa.Column('phone', sa.String(length=45), nullable=True),
    sa.Column('userStatus', sa.Enum('inactive', 'active', name='userstatus'), nullable=False),
    sa.Column('service_centerid', sa.Integer(), nullable=True),
```

```
    sa.Column('roles', sa.String(length=32), nullable=False),
    sa.ForeignKeyConstraint(['service_centerid'], ['service_center.serviceid'], ondelete='CASCADE'),
    sa.PrimaryKeyConstraint('userid'),
    sa.UniqueConstraint('username')
    )
    op.create_table('device',
    sa.Column('deviceid', sa.Integer(), nullable=False),
    sa.Column('firm', sa.String(length=45), nullable=False),
    sa.Column('model', sa.String(length=45), nullable=False),
    sa.Column('status', sa.Enum('arrived', 'in_work', 'ready', name='devicestatus'), nullable=True),
    sa.Column('userid', sa.Integer(), nullable=True),
    sa.ForeignKeyConstraint(['userid'], ['user.userid'], ondelete='CASCADE'),
    sa.PrimaryKeyConstraint('deviceid')
    )
    op.create_table('problem_to_device',
    sa.Column('problemid', sa.Integer(), nullable=True),
    sa.Column('deviceid', sa.Integer(), nullable=True),
    sa.ForeignKeyConstraint(['deviceid'], ['device.deviceid'], ondelete='CASCADE'),
    sa.ForeignKeyConstraint(['problemid'], ['problem.problemid'], ondelete='CASCADE')
    )
    # ### end Alembic commands ###


def downgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    op.drop_table('problem_to_device')
    op.drop_table('device')
    op.drop_table('user')
    op.drop_table('service_center')
    op.drop_table('problem')
    # ### end Alembic commands ###
```

eec1f10c1d5d_create_model.py

```
"""Create model

Revision ID: eec1f10c1d5d
Revises: 0bd36dbb9974
Create Date: 2022-11-26 18:53:00.440321

"""
from alembic import op
import sqlalchemy as sa


# revision identifiers, used by Alembic.
revision = 'eec1f10c1d5d'
down_revision = '0bd36dbb9974'
branch_labels = None
depends_on = None
```

```
def upgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    op.alter_column('user', 'password',
               existing_type=sa.VARCHAR(length=45),
               type_=sa.String(length=128),
               existing_nullable=False)
    # ### end Alembic commands ###


def downgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    op.alter_column('user', 'password',
               existing_type=sa.String(length=128),
               type_=sa.VARCHAR(length=45),
               existing_nullable=False)
    # ### end Alembic commands ###
```

/alembic/migrations:

env.py

```
from logging.config import fileConfig

from sqlalchemy import engine_from_config
from sqlalchemy import pool
from models import Base



from alembic import context

target_metadata=Base.metadata
# this is the Alembic Config object, which provides
# access to the values within the .ini file in use.
config = context.config

# Interpret the config file for Python logging.
# This line sets up loggers basically.
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

# add your model's MetaData object here
# for 'autogenerate' support
# from myapp import mymodel
# target_metadata = mymodel.Base.metadata



# other values from the config, defined by the needs of env.py,
# can be acquired:
# my_important_option = config.get_main_option("my_important_option")
# ... etc.


def run_migrations_offline() -> None:
    """Run migrations in 'offline' mode.

    This configures the context with just a URL
```

```
    and not an Engine, though an Engine is acceptable
    here as well.  By skipping the Engine creation
    we don't even need a DBAPI to be available.

    Calls to context.execute() here emit the given string to the
    script output.

    """
    url = config.get_main_option("sqlalchemy.url")
    context.configure(
        url=url,
        target_metadata=target_metadata,
        literal_binds=True,
        compare_type=True,
        dialect_opts={"paramstyle": "named"},
    )

    with context.begin_transaction():
        context.run_migrations()


def run_migrations_online() -> None:
    """Run migrations in 'online' mode.

    In this scenario we need to create an Engine
    and associate a connection with the context.

    """
    connectable = engine_from_config(
        config.get_section(config.config_ini_section),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    with connectable.connect() as connection:
        context.configure(
            connection=connection, target_metadata=target_metadata, compare_type=True
        )

        with context.begin_transaction():
            context.run_migrations()


if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()
```

script.py.mako

```
"""${message}

Revision ID: ${up_revision}
Revises: ${down_revision | comma,n}
Create Date: ${create_date}

"""
from alembic import op
import sqlalchemy as sa
${imports if imports else ""}

# revision identifiers, used by Alembic.
```

```
revision = ${repr(up_revision)}
down_revision = ${repr(down_revision)}
branch_labels = ${repr(branch_labels)}
depends_on = ${repr(depends_on)}


def upgrade() -> None:
    ${upgrades if upgrades else "pass"}


def downgrade() -> None:
    ${downgrades if downgrades else "pass"}
```

/alembic

__init__.py

alembic.ini

```
# A generic, single database configuration.

[alembic]
# path to migration scripts
script_location = migrations

# template used to generate migration file names; The default value is %%(rev)s_%%(slug)s
# Uncomment the line below if you want the files to be prepended with date and time
# see https://alembic.sqlalchemy.org/en/latest/tutorial.html#editing-the-ini-file
# for all available tokens
# file_template = %%(year)d_%%(month).2d_%%(day).2d_%%(hour).2d%%(minute).2d-%%(rev)s_%%(slug)s

# sys.path path, will be prepended to sys.path if present.
# defaults to the current working directory.
prepend_sys_path = .

# timezone to use when rendering the date within the migration file
# as well as the filename.
# If specified, requires the python-dateutil library that can be
# installed by adding `alembic[tz]` to the pip requirements
# string value is passed to dateutil.tz.gettz()
# leave blank for localtime
# timezone =

# max length of characters to apply to the
# "slug" field
# truncate_slug_length = 40

# set to 'true' to run the environment during
# the 'revision' command, regardless of autogenerate
# revision_environment = false

# set to 'true' to allow .pyc and .pyo files without
# a source .py file to be detected as revisions in the
# versions/ directory
# sourceless = false

# version location specification; This defaults
# to migrations/versions.  When using multiple version
# directories, initial revisions must be specified with --version-path.
# The path separator used here should be the separator specified by "version_path_separator" below.
# version_locations = %(here)s/bar:%(here)s/bat:migrations/versions

# version path separator; As mentioned above, this is the character used to split
# version_locations. The default within new alembic.ini files is "os", which uses os.pathsep.
```

```
# If this key is omitted entirely, it falls back to the legacy behavior of splitting on spaces and/or commas.
# Valid values for version_path_separator are:
#
# version_path_separator = :
# version_path_separator = ;
# version_path_separator = space
version_path_separator = os  # Use os.pathsep. Default configuration used for new projects.

# the output encoding used when revision files
# are written from script.py.mako
# output_encoding = utf-8

sqlalchemy.url = postgresql+psycopg2://postgres:2580@localhost/service_center


[post_write_hooks]
# post_write_hooks defines scripts or Python functions that are run
# on newly generated revision scripts.  See the documentation for further
# detail and examples

# format using "black" - use the console_scripts runner, against the "black" entrypoint
# hooks = black
# black.type = console_scripts
# black.entrypoint = black
# black.options = -l 79 REVISION_SCRIPT_FILENAME

# Logging configuration
[loggers]
keys = root,sqlalchemy,alembic

[handlers]
keys = console

[formatters]
keys = generic

[logger_root]
level = WARN
handlers = console
qualname =

[logger_sqlalchemy]
level = WARN
handlers =
qualname = sqlalchemy.engine

[logger_alembic]
level = INFO
handlers =
qualname = alembic

[handler_console]
class = StreamHandler
args = (sys.stderr,)
level = NOTSET
formatter = generic
[formatter_generic]
format = %(levelname)-5.5s [%(name)s] %(message)s
datefmt = %H:%M:%S
```

checkModels.py

```
from models import Device, User, Service_Center, Problem, problem_to_device
from database import db_session

device = Device(deviceid=1, firm='Apple', model='IPHONE 10', userid=1)
device1 = Device(deviceid=2, firm='Samsung', model='Galaxy S21', userid=2)
user = User(userid=1, username='user123', lastname='Kovaliv', email='kovaliv@gmail.com', password='45765')
user1 = User(userid=2, username='user000', lastname='Vasyliv', email='vas356@gmail.com', password='876765')
```

```
service = Service_Center(serviceid=1, city='Kalush', adress='ertyu678', phone='098463893', rating='3.8')
service1 = Service_Center(serviceid=2, city='Kalush', adress='998', phone='096398793', rating='4.8')
problem = Problem(problemid=1, branch='os', daysToSolve=5)
problem1 = Problem(problemid=2, branch='hardware', daysToSolve=9)
device1.problems.append(problem)
device1.problems.append(problem1)



db_session.add(device)
db_session.add(device1)
db_session.add(user)
db_session.add(user1)
db_session.add(service)
db_session.add(service1)



db_session.commit()
```

databese.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import scoped_session, sessionmaker
from sqlalchemy.ext.declarative import declarative_base

DBURL = 'postgresql+psycopg2://postgres:2580@localhost/service_center'
engine=create_engine(DBURL)
db_session = scoped_session(sessionmaker(autocommit=False,
                                         autoflush=False,
                                         bind=engine))
Base = declarative_base()
Base.query = db_session.query_property()

def init_db():
    from .models import Service_Center, User, Problem, Device
    Base.metadata.create_all(bind=engine)
```

models.py

```
from sqlalchemy import create_engine, Column, Integer, String, Enum, orm, ForeignKey, Table, Enum
from sqlalchemy.orm import sessionmaker, scoped_session, declarative_base, relationship
from .database import Base
import enum


class userStatus(enum.Enum):
    inactive = 0
    active = 1


class deviceStatus(enum.Enum):
    arrived = 0
    in_work = 1
    ready = 2


class branchEnum(enum.Enum):
    screen = 0
    battery = 1
    button = 2
    os = 3
    hardware = 4
    other = 5


class Service_Center(Base):
```

```
    __tablename__ = "service_center"
    serviceid = Column(Integer, primary_key=True)
    city = Column(String(45), nullable=False)
    adress = Column(String(45), nullable=False)
    phone = Column(String(45), nullable=False)
    rating = Column(String(45), nullable=True)

    def __repr__(self):
        return f'Service_Center(serviceid={self.serviceid},city={self.city},address={self.adress})'


class User(Base):
    __tablename__ = "user"
    userid = Column(Integer, primary_key=True)
    username = Column(String(45), nullable=False, unique=True)
    firstname = Column(String(45), nullable=True)
    lastname = Column(String(45), nullable=False)
    email = Column(String(45), nullable=False)
    password = Column(String(128), nullable=False)
    phone = Column(String(45), nullable=True)
    userStatus = Column(Enum(userStatus), default=userStatus.active, nullable=False)
    service_centerid = Column(Integer, ForeignKey(Service_Center.serviceid, ondelete="CASCADE"))
    service_center = orm.relationship(Service_Center, backref="owner")
    role = Column(String(32), default="client", nullable=False)

    def __repr__(self):
        return f'User(username={self.username},name={self.firstname + " " if self.firstname else ""}{self.lastname})'

class Problem(Base):
    __tablename__ = "problem"
    problemid = Column(Integer, primary_key=True)
    branch = Column(Enum(branchEnum), nullable=False)
    daysToSolve = Column(Integer, nullable=True)

    def __repr__(self):
        return f'Problem(problemid={self.problemid})'
class Device(Base):
    __tablename__ = "device"
    deviceid = Column(Integer, primary_key=True)
    firm = Column(String(45), nullable=False)
    model = Column(String(45), nullable=False)
    status = Column(Enum(deviceStatus), nullable=True)
    userid = Column(Integer, ForeignKey(User.userid, ondelete="CASCADE"))
    user = orm.relationship(User, backref="device")
    problems = relationship("Problem", secondary='problem_to_device', passive_deletes=True)

    def __repr__(self):
        return f'Device(deviceid={self.deviceid},firm={self.firm},model={self.model})'
problem_to_device = Table(
    "problem_to_device",
    Base.metadata,
    Column("problemid", ForeignKey("problem.problemid", ondelete="CASCADE")),
    Column("deviceid", ForeignKey("device.deviceid", ondelete="CASCADE"))
)
```

/LabySpp

custom.py

```
def json_error(msg, code):
    return {'error': {'code': code, 'message' : msg}}, code



class errs:
    not_found = json_error('Not found', 404)
    bad_request = json_error('Invalid request. Bad request body', 400)
    exists = json_error('Forbidden. Already exists', 403)
```

deviceapi.py

```
from flask import request
from flask_restful import Resource
from alembic.database import db_session
from alembic.models import Service_Center, User, Problem, Device, deviceStatus
from schemas import DeviceSchema, ProblemSchema
from marshmallow.exceptions import ValidationError
from custom import json_error, errs
from main import bcrypt, master_required, admin_required, get_user
from flask_jwt_extended import jwt_required
from flask_jwt_extended import create_access_token

device_schema = DeviceSchema()
problem_schema = ProblemSchema()


class DevicesAPI(Resource):
    @admin_required()
    def get(self):
        devices = Device.query.all()
        return device_schema.dump(devices, many=True), 200

    @master_required()
    def post(self):
        json_data = request.get_json()
        if not json_data:
            return errs.bad_request
        device = Device.query.get(json_data.get('deviceid', None))
        if device:
            return errs.exists
        if not 'userid' in json_data:
            return json_error('Invalid request. No userid provided', 400)
        user = User.query.get(json_data.get('userid', None))
        if not user:
            return json_error('Invalid request. User not found', 400)
        claims = get_user()
        if claims.role != 'admin' and claims.service_centerid != user.service_centerid:
            return json_error("Forbidden. Master can add devices only of his clients.", 403)
        try:
            data = device_schema.load(json_data, session=db_session)
        except ValidationError as err:
            return json_error(err.messages, 400)
        db_session.add(data)
        db_session.commit()

        return {"message": "Created device.", "device": json_data}

    @master_required()
    def put(self):
        json_data = request.get_json()
        if not json_data:
            return errs.bad_request
        device = Device.query.get(json_data.get('deviceid', None))
```

```
        if not device:
            return errs.not_found
        claims = get_user()
        if claims.role != 'admin' and claims.service_centerid != device.user.service_centerid:
            return json_error("Forbidden. Master can change devices only of his clients.", 403)
        user = User.query.get(json_data.get('userid', None))
        if not user:
            return json_error('Invalid request. User not found', 400)
        claims = get_user()
        if claims.role != 'admin' and claims.service_centerid != user.service_centerid:
            return json_error("Forbidden. Master can assign devices only to his clients.", 403)
        try:
            data = device_schema.load(json_data, session=db_session)
        except ValidationError as err:
            return json_error(err.messages, 400)
        db_session.add(data)
        db_session.commit()

        return {"message": "Updated device.", "device": json_data}


class DeviceStatusAPI(Resource):
    @admin_required()
    def get(self):
        json_data = request.get_json()
        if not json_data:
            return errs.bad_request
        status = json_data.get('status', None)
        deviceStatuses = [e.name for e in deviceStatus]
        if status not in deviceStatuses and status is not None:
            return json_error(
                'Invalid request. Bad status value. Must be on of: ' + ', '.join(deviceStatuses) + ' or null', 400)
        devices = Device.query.filter_by(status=status)
        return device_schema.dump(devices, many=True), 200


class DeviceFirmAPI(Resource):
    @admin_required()
    def get(self):
        json_data = request.get_json()
        if not json_data:
            return errs.bad_request
        if len(json_data) != 1 or 'firm' not in json_data:
            return json_error('Only firm field must be provided', 400)
        firm = json_data['firm']

        devices = Device.query.filter_by(firm=firm)
        return device_schema.dump(devices, many=True), 200


class DeviceAPI(Resource):
    @jwt_required()
    def get(self, deviceid):
        device = Device.query.get(deviceid)
        if not device:
            return errs.not_found
        claims = get_user()
        if claims.role != 'admin' and claims.service_centerid != device.user.service_centerid:
            return json_error("Forbidden. User can get info about his devices.", 403)
        return device_schema.dump(device), 200

    @master_required()
    def delete(self, deviceid):
        device = Device.query.get(deviceid)
        if not device:
            return errs.not_found
        claims = get_user()
        if claims.role != 'admin' and claims.service_centerid != device.user.service_centerid:
```

```
            return json_error("Forbidden. Master can delete devices only from his clients.", 403)
        db_session.delete(device)
        db_session.commit()
        return '', 204


class DeviceProblemsAPI(Resource):
    @jwt_required()
    def get(self, deviceid):
        device = Device.query.get(deviceid)
        if not device:
            return errs.not_found
        claims = get_user()
        if claims.role != 'admin' and claims.service_centerid != device.user.service_centerid:
            return json_error("Forbidden. User can get info about problems only of his devices.", 403)
        return {'device': device_schema.dump(device), 'problems': problem_schema.dump(device.problems, many=True)}, 200

    @master_required()
    def post(self, deviceid):
        device = Device.query.get(deviceid)
        if not device:
            return errs.not_found
        claims = get_user()
        if claims.role != 'admin' and claims.service_centerid != device.user.service_centerid:
            return json_error("Forbidden. Master can add problems only to his client's devices.", 403)
        json_data = request.get_json()
        if not json_data:
            return errs.bad_request
        if 'problemid' not in json_data:
            return errs.bad_request
        problem = Problem.query.get(json_data.get('problemid', None))
        if problem:
            device.problems.append(problem)

        try:
            data = problem_schema.load(json_data, session=db_session)
        except ValidationError as err:
            return json_error(err.messages, 400)
        device.problems.append(data)
        db_session.add(data)
        db_session.commit()
        return {'device': device_schema.dump(device), 'Added new problem': json_data}, 200


class DeviceProblemAPI(Resource):
    @jwt_required()
    def get(self, deviceid, problemid):
        device = Device.query.get(deviceid)
        if not device:
            return json_error('Device not found', 404)
        claims = get_user()
        if claims.role != 'admin' and claims.service_centerid != device.user.service_centerid:
            return json_error("Forbidden. User can get info only about his problems.", 403)
        problem = None
        for pr in device.problems:
            if pr.problemid == problemid:
                problem = pr

        if not problem:
            return json_error('Problem not found', 404)
        return {'device': device_schema.dump(device), 'problem': problem_schema.dump(problem)}, 200

    @master_required()
    def delete(self, deviceid, problemid):
        device = Device.query.get(deviceid)
        if not device:
            return json_error('Device not found', 404)
        claims = get_user()
```

```
        if claims.role != 'admin' and claims.service_centerid != device.user.service_centerid:
            return json_error("Forbidden. Only master can delete only problems of his devices.", 403)
        problem = None
        for pr in device.problems:
            if pr.problemid == problemid:
                problem = pr

        if not problem:
            return json_error('Problem not found', 404)
        db_session.delete(problem)
        db_session.commit()
        return '', 204

    @master_required()
    def put(self, deviceid, problemid):
        device = Device.query.get(deviceid)
        if not device:
            return json_error('Device not found', 404)
        claims = get_user()
        if claims.role != 'admin' and claims.service_centerid != device.user.service_centerid:
            return json_error("Forbidden. Only master can update only problems of his devices.", 403)
        problem = Problem.query.get(problemid)
        if not problem:
            return json_error('Problem not found', 404)
        device.problems.append(problem)
        db_session.commit()
        return {'device': device_schema.dump(device), 'Added existing problem': problem_schema.dump(problem)}, 200
```

main.py

```
from flask import Flask
from flask import jsonify
from flask_restful import Api
from alembic.database import db_session, init_db
from flask_bcrypt import Bcrypt
from custom import json_error

app = Flask(__name__)
api = Api(app, prefix='/api/v1')
bcrypt = Bcrypt(app)

init_db()

from flask_jwt_extended import get_jwt
from flask_jwt_extended import JWTManager
from flask_jwt_extended import verify_jwt_in_request

app.config["JWT_SECRET_KEY"] = "super-secret"
jwt = JWTManager(app)

from functools import wraps
from alembic.models import User

def get_user():
    return User.query.filter_by(username=get_jwt()['sub'])[0]

def admin_required():
    def wrapper(fn):
        @wraps(fn)
        def decorator(*args, **kwargs):
            verify_jwt_in_request()
            claims = get_jwt()
            user = User.query.filter_by(username=claims['sub'])[0]
            if not user:
                return json_error("Authentication failed. User not found", 404)
```

```
            if user.role == 'admin':
                return fn(*args, **kwargs)
            return json_error("Must be an admin", 403)
        return decorator

    return wrapper


def master_required():
    def wrapper(fn):
        @wraps(fn)
        def decorator(*args, **kwargs):
            verify_jwt_in_request()
            claims = get_jwt()
            user = User.query.filter_by(username=claims['sub'])[0]
            if not user:
                return json_error("Authentication failed. User not found.", 404)
            if user.role == 'master' or user.role == 'admin':
                return fn(*args, **kwargs)
            return json_error("Must be a master.", 403)
        return decorator

    return wrapper


@app.teardown_appcontext
def shutdown_session(exception=None):
    db_session.remove()


from deviceapi import DevicesAPI, DeviceAPI, DeviceStatusAPI, DeviceFirmAPI, DeviceProblemAPI, DeviceProblemsAPI
from serviceapi import ServicesAPI, ServiceAPI, ServiceDeviceAPI
from userapi import UsersAPI, UserAPI, UserDeviceAPI, UserLoginAPI
from problemapi import ProblemsAPI, ProblemAPI, ProblemBranchAPI

api.add_resource(DevicesAPI, '/device')
api.add_resource(DeviceAPI, '/device/<int:deviceid>')
api.add_resource(DeviceProblemsAPI, '/device/<int:deviceid>/problem')
api.add_resource(DeviceProblemAPI, '/device/<int:deviceid>/problem/<int:problemid>')
api.add_resource(DeviceStatusAPI, '/device/findByStatus')
api.add_resource(DeviceFirmAPI, '/device/findByFirm')

api.add_resource(ServicesAPI, '/service')
api.add_resource(ServiceDeviceAPI, '/service/device')
api.add_resource(ServiceAPI, '/service/<int:serviceid>')

api.add_resource(UsersAPI, '/user')
api.add_resource(UserAPI, '/user/<username>')
api.add_resource(UserDeviceAPI, '/user/<username>/device')
api.add_resource(UserLoginAPI, '/login')

api.add_resource(ProblemsAPI, '/problem')
api.add_resource(ProblemAPI, '/problem/<int:problemid>')
api.add_resource(ProblemBranchAPI, '/problem/findByBranch')
```

Pipfile

```
[[source]]
url = "https://pypi.org/simple"
verify_ssl = true
name = "pypi"

[packages]
requests = "*"
psycopg2="*"
```

```
alembic = "*"
sqlalchemy = "*"
flask = "*"
waitress = "*"
flask-restful = "*"
marshmallow-sqlalchemy = "*"
marshmallow-enum = "*"
Flask-JWT-Extended="*"
flask_bcrypt="*"


[dev-packages]

[requires]
python_version = "3.8"
python_full_version = "3.8.9"
```

Pipfile.lock


```
{
    "_meta": {
        "hash": {
            "sha256": "b2a872392f5db768b923bd6ff6abdc33d732d6ccb3720753a36545e1186454c3"
        },
        "pipfile-spec": 6,
        "requires": {
            "python_full_version": "3.8.9",
            "python_version": "3.8"
        },
        "sources": [
            {
                "name": "pypi",
                "url": "https://pypi.org/simple",
                "verify_ssl": true
            }
        ]
    },
    "default": {
        "alembic": {
            "hashes": [
                "sha256:0a024d7f2de88d738d7395ff866997314c837be6104e90c5724350313dee4da4",
                "sha256:cd0b5e45b14b706426b833f06369b9a6d5ee03f826ec3238723ce8caaf6e5ffa"
            ],
            "index": "pypi",
            "version": "==1.8.1"
        },
        "aniso8601": {
            "hashes": [
                "sha256:1d2b7ef82963909e93c4f24ce48d4de9e66009a21bf1c1e1c85bdd0812fe412f",
                "sha256:72e3117667eedf66951bb2d93f4296a56b94b078a8a95905a052611fb3f1b973"
            ],
            "version": "==9.0.1"
        },
        "bcrypt": {
            "hashes": [
                "sha256:089098effa1bc35dc055366740a067a2fc76987e8ec75349eb9484061c54f535",
                "sha256:08d2947c490093a11416df18043c27abe3921558d2c03e2076ccb28a116cb6d0",
                "sha256:0eaa47d4661c326bfc9d08d16debbc4edf78778e6aaba29c1bc7ce67214d4410",
                "sha256:27d375903ac8261cfe4047f6709d16f7d18d39b1ec92aaf72af989552a650ebd",
                "sha256:2b3ac11cf45161628f1f3733263e63194f22664bf4d0c0f3ab34099c02134665",
                "sha256:2caffdae059e06ac23fce178d31b4a702f2a3264c20bfb5ff541b338194d8fab",
                "sha256:3100851841186c25f127731b9fa11909ab7b1df6fc4b9f8353f4f1fd952fbf71",
                "sha256:5ad4d32a28b80c5fa6671ccfb43676e8c1cc232887759d1cd7b6f56ea4355215",
                "sha256:67a97e1c405b24f19d08890e7ae0c4f7ce1e56a712a016746c8b2d7732d65d4b",
```

```
        "sha256:705b2cea8a9ed3d55b4491887ceadb0106acf7c6387699fca771af56b1cdeeda",
        "sha256:8a68f4341daf7522fe8d73874de8906f3a339048ba406be6ddc1b3ccb16fc0d9",
        "sha256:a522427293d77e1c29e303fc282e2d71864579527a04ddcfda6d4f8396c6c36a",
        "sha256:ae88eca3024bb34bb3430f964beab71226e761f51b912de5133470b649d82344",
        "sha256:b1023030aec778185a6c16cf70f359cbb6e0c289fd564a7cfa29e727a1c38f8f",
        "sha256:b3b85202d95dd568efcb35b53936c5e3b3600c7cdcc6115ba461df3a8e89f38d",
        "sha256:b57adba8a1444faf784394de3436233728a1ecaeb6e07e8c22c8848f179b893c",
        "sha256:bf4fa8b2ca74381bb5442c089350f09a3f17797829d958fad058d6e44d9eb83c",
        "sha256:ca3204d00d3cb2dfed07f2d74a25f12fc12f73e606fcaa6975d1f7ae69cacbb2",
        "sha256:cbb03eec97496166b704ed663a53680ab57c5084b2fc98ef23291987b525cb7d",
        "sha256:e9a51bbfe7e9802b5f3508687758b564069ba937748ad7b9e890086290d2f79e",
        "sha256:fbdaec13c5105f0c4e5c52614d04f0bca5f5af007910daa8b6b12095edaa67b3"
      ],
      "markers": "python_version >= '3.6'",
      "version": "==4.0.1"
    },
    "certifi": {
      "hashes": [
        "sha256:0d9c601124e5a6ba9712dbc60d9c53c21e34f5f641fe83002317394311bdce14",
        "sha256:90c1a32f1d68f940488354e36370f6cca89f0f106db09518524c88d6ed83f382"
      ],
      "markers": "python_version >= '3.6'",
      "version": "==2022.9.24"
    },
    "charset-normalizer": {
      "hashes": [
        "sha256:5a3d016c7c547f69d6f81fb0db9449ce888b418b5b9952cc5e6e66843e9dd845",
        "sha256:83e9a75d1911279afd89352c68b45348559d1fc0506b054b346651b5e7fee29f"
      ],
      "markers": "python_full_version >= '3.6.0'",
      "version": "==2.1.1"
    },
    "click": {
      "hashes": [
        "sha256:7682dc8afb30297001674575ea00d1814d808d6a36af415a82bd481d37ba7b8e",
        "sha256:bb4d8133cb15a609f44e8213d9b391b0809795062913b383c62be0ee95b1db48"
      ],
      "markers": "python_version >= '3.7'",
      "version": "==8.1.3"
    },
    "colorama": {
      "hashes": [
        "sha256:08695f5cb7ed6e0531a20572697297273c47b8cae5a63ffc6d6ed5c201be6e44",
        "sha256:4f1d9991f5acc0ca119f9d443620b77f9d6b33703e51011c16baf57afb285fc6"
      ],
      "markers": "platform_system == 'Windows'",
      "version": "==0.4.6"
    },
    "flask": {
      "hashes": [
        "sha256:642c450d19c4ad482f96729bd2a8f6d32554aa1e231f4f6b4e7e5264b16cca2b",
        "sha256:b9c46cc36662a7949f34b52d8ec7bb59c0d74ba08ba6cb9ce9adc1d8676d9526"
      ],
      "index": "pypi",
      "version": "==2.2.2"
    },
    "flask-bcrypt": {
      "hashes": [
        "sha256:062fd991dc9118d05ac0583675507b9fe4670e44416c97e0e6819d03d01f808a",
        "sha256:f07b66b811417ea64eb188ae6455b0b708a793d966e1a80ceec4a23bc42a4369"
      ],
      "index": "pypi",
      "version": "==1.0.1"
    },
    "flask-httpauth": {
      "hashes": [
        "sha256:a237e4c8ec1d339298a0eb88e5af2ca36117949b621631649462800e57e1ba37",
        "sha256:f7199e7bad20d5b68b3f0b62bddfca7637c55087e9d02f605ae26e0de479fd94"
```

```
    ],
    "index": "pypi",
    "version": "==4.7.0"
  },
  "flask-jwt": {
    "hashes": [
      "sha256:49c0672fbde0f1cd3374bd834918d28956e3c521c7e00089cdc5380d323bd0ad"
    ],
    "index": "pypi",
    "version": "==0.3.2"
  },
  "flask-jwt-extended": {
    "hashes": [
      "sha256:62b521d75494c290a646ae8acc77123721e4364790f1e64af0038d823961fbf0",
      "sha256:a85eebfa17c339a7260c4643475af444784ba6de5588adda67406f0a75599553"
    ],
    "index": "pypi",
    "version": "==4.4.4"
  },
  "flask-restful": {
    "hashes": [
      "sha256:4970c49b6488e46c520b325f54833374dc2b98e211f1b272bd4b0c516232afe2",
      "sha256:ccec650b835d48192138c85329ae03735e6ced58e9b2d9c2146d6c84c06fa53e"
    ],
    "index": "pypi",
    "version": "==0.3.9"
  },
  "greenlet": {
    "hashes": [
      "sha256:0109af1138afbfb8ae647e31a2b1ab030f58b21dd8528c27beaeb0093b7938a9",
      "sha256:0459d94f73265744fee4c2d5ec44c6f34aa8a31017e6e9de770f7bcf29710be9",
      "sha256:04957dc96669be041e0c260964cfef4c77287f07c40452e61abe19d647505581",
      "sha256:0722c9be0797f544a3ed212569ca3fe3d9d1a1b13942d10dd6f0e8601e484d26",
      "sha256:097e3dae69321e9100202fc62977f687454cd0ea147d0fd5a766e57450c569fd",
      "sha256:0b493db84d124805865adc587532ebad30efa68f79ad68f11b336e0a51ec86c2",
      "sha256:13ba6e8e326e2116c954074c994da14954982ba2795aebb881c07ac5d093a58a",
      "sha256:13ebf93c343dd8bd010cd98e617cb4c1c1f352a0cf2524c82d3814154116aa82",
      "sha256:1407fe45246632d0ffb7a3f4a520ba4e6051fc2cbd61ba1f806900c27f47706a",
      "sha256:1bf633a50cc93ed17e494015897361010fc08700d92676c87931d3ea464123ce",
      "sha256:2d0bac0385d2b43a7bd1d651621a4e0f1380abc63d6fb1012213a401cbd5bf8f",
      "sha256:3001d00eba6bbf084ae60ec7f4bb8ed375748f53aeaefaf2a37d9f0370558524",
      "sha256:356e4519d4dfa766d50ecc498544b44c0249b6de66426041d7f8b751de4d6b48",
      "sha256:38255a3f1e8942573b067510f9611fc9e38196077b0c8eb7a8c795e105f9ce77",
      "sha256:3d75b8d013086b08e801fbbb896f7d5c9e6ccd44f13a9241d2bf7c0df9eda928",
      "sha256:41b825d65f31e394b523c84db84f9383a2f7eefc13d987f308f4663794d2687e",
      "sha256:42e602564460da0e8ee67cb6d7236363ee5e131aa15943b6670e44e5c2ed0f67",
      "sha256:4aeaebcd91d9fee9aa768c1b39cb12214b30bf36d2b7370505a9f2165fedd8d9",
      "sha256:4c8b1c43e75c42a6cafcc71defa9e01ead39ae80bd733a2608b297412beede68",
      "sha256:4d37990425b4687ade27810e3b1a1c37825d242ebc275066cfee8cb6b8829ccd",
      "sha256:4f09b0010e55bec3239278f642a8a506b91034f03a4fb28289a7d448a67f1515",
      "sha256:505138d4fa69462447a562a7c2ef723c6025ba12ac04478bc1ce2fcc279a2db5",
      "sha256:5067920de254f1a2dee8d3d9d7e4e03718e8fd2d2d9db962c8c9fa781ae82a39",
      "sha256:56961cfca7da2fdd178f95ca407fa330c64f33289e1804b592a77d5593d9bd94",
      "sha256:5a8e05057fab2a365c81abc696cb753da7549d20266e8511eb6c9d9f72fe3e92",
      "sha256:659f167f419a4609bc0516fb18ea69ed39dbb25594934bd2dd4d0401660e8a1e",
      "sha256:662e8f7cad915ba75d8017b3e601afc01ef20deeeabf281bd00369de196d7726",
      "sha256:6f61d71bbc9b4a3de768371b210d906726535d6ca43506737682caa754b956cd",
      "sha256:72b00a8e7c25dcea5946692a2485b1a0c0661ed93ecfedfa9b6687bd89a24ef5",
      "sha256:811e1d37d60b47cb8126e0a929b58c046251f28117cb16fcd371eed61f66b764",
      "sha256:81b0ea3715bf6a848d6f7149d25bf018fd24554a4be01fcbbe3fdc78e890b955",
      "sha256:88c8d517e78acdf7df8a2134a3c4b964415b575d2840a2746ddb1cc6175f8608",
      "sha256:8dca09dedf1bd8684767bc736cc20c97c29bc0c04c413e3276e0962cd7aeb148",
      "sha256:974a39bdb8c90a85982cdb78a103a32e0b1be986d411303064b28a80611f6e51",
      "sha256:9e112e03d37987d7b90c1e98ba5e1b59e1645226d78d73282f45b326f7bddcb9",
      "sha256:9e9744c657d896c7b580455e739899e492a4a452e2dd4d2b3e459f6b244a638d",
      "sha256:9ed358312e63bf683b9ef22c8e442ef6c5c02973f0c2a939ec1d7b50c974015c",
      "sha256:9f2c221eecb7ead00b8e3ddb913c67f75cba078fd1d326053225a3f59d850d72",
      "sha256:a20d33124935d27b80e6fdacbd34205732660e0a1d35d8b10b3328179a2b51a1",
```

```
            "sha256:a4c0757db9bd08470ff8277791795e70d0bf035a011a528ee9a5ce9454b6cba2",
            "sha256:afe07421c969e259e9403c3bb658968702bc3b78ec0b6fde3ae1e73440529c23",
            "sha256:b1992ba9d4780d9af9726bbcef6a1db12d9ab1ccc35e5773685a24b7fb2758eb",
            "sha256:b23d2a46d53210b498e5b701a1913697671988f4bf8e10f935433f6e7c332fb6",
            "sha256:b5e83e4de81dcc9425598d9469a624826a0b1211380ac444c7c791d4a2137c19",
            "sha256:be35822f35f99dcc48152c9839d0171a06186f2d71ef76dc57fa556cc9bf6b45",
            "sha256:be9e0fb2ada7e5124f5282d6381903183ecc73ea019568d6d63d33f25b2a9000",
            "sha256:c140e7eb5ce47249668056edf3b7e9900c6a2e22fb0eaf0513f18a1b2c14e1da",
            "sha256:c6a08799e9e88052221adca55741bf106ec7ea0710bca635c208b751f0d5b617",
            "sha256:cb242fc2cda5a307a7698c93173d3627a2a90d00507bccf5bc228851e8304963",
            "sha256:cce1e90dd302f45716a7715517c6aa0468af0bf38e814ad4eab58e88fc09f7f7",
            "sha256:cd4ccc364cf75d1422e66e247e52a93da6a9b73cefa8cad696f3cbbb75af179d",
            "sha256:d21681f09e297a5adaa73060737e3aa1279a13ecdcfcc6ef66c292cb25125b2d",
            "sha256:d38ffd0e81ba8ef347d2be0772e899c289b59ff150ebbbbe05dc61b1246eb4e0",
            "sha256:d566b82e92ff2e09dd6342df7e0eb4ff6275a3f08db284888dcd98134dbd4243",
            "sha256:d5b0ff9878333823226d270417f24f4d06f235cb3e54d1103b71ea537a6a86ce",
            "sha256:d6ee1aa7ab36475035eb48c01efae87d37936a8173fc4d7b10bb02c2d75dd8f6",
            "sha256:db38f80540083ea33bdab614a9d28bcec4b54daa5aff1668d7827a9fc769ae0a",
            "sha256:ea688d11707d30e212e0110a1aac7f7f3f542a259235d396f88be68b649e47d1",
            "sha256:f6327b6907b4cb72f650a5b7b1be23a2aab395017aa6f1adb13069d66360eb3f",
            "sha256:fb412b7db83fe56847df9c47b6fe3f13911b06339c2aa02dcc09dce8bbf582cd"
        ],
        "markers": "python_version >= '3' and platform_machine == 'aarch64' or (platform_machine == 'ppc64le' or (platform_machine == 'x86_64' or
(platform_machine == 'amd64' or (platform_machine == 'AMD64' or (platform_machine == 'win32' or platform_machine == 'WIN32')))))",
        "version": "==2.0.1"
    },
    "idna": {
        "hashes": [
            "sha256:814f528e8dead7d329833b91c5faa87d60bf71824cd12a7530b5526063d02cb4",
            "sha256:90b77e79eaa3eba6de819a0c442c0b4ceefc341a7a2ab77d7562bf49f425c5c2"
        ],
        "markers": "python_version >= '3.5'",
        "version": "==3.4"
    },
    "importlib-metadata": {
        "hashes": [
            "sha256:da31db32b304314d044d3c12c79bd59e307889b287ad12ff387b3500835fc2ab",
            "sha256:ddb0e35065e8938f867ed4928d0ae5bf2a53b7773871bfe6bcc7e4fcdc7dea43"
        ],
        "markers": "python_version < '3.9'",
        "version": "==5.0.0"
    },
    "importlib-resources": {
        "hashes": [
            "sha256:c01b1b94210d9849f286b86bb51bcea7cd56dde0600d8db721d7b81330711668",
            "sha256:ee17ec648f85480d523596ce49eae8ead87d5631ae1551f913c0100b5edd3437"
        ],
        "markers": "python_version < '3.9'",
        "version": "==5.10.0"
    },
    "itsdangerous": {
        "hashes": [
            "sha256:2c2349112351b88699d8d4b6b075022c0808887cb7ad10069318a8b0bc88db44",
            "sha256:5dbbc68b317e5e42f327f9021763545dc3fc3bfe22e6deb96aaf1fc38874156a"
        ],
        "markers": "python_version >= '3.7'",
        "version": "==2.1.2"
    },
    "jinja2": {
        "hashes": [
            "sha256:31351a702a408a9e7595a8fc6150fc3f43bb6bf7e319770cbc0db9df9437e852",
            "sha256:6088930bfe239f0e6710546ab9c19c9ef35e29792895fed6e6e31a023a182a61"
        ],
        "markers": "python_version >= '3.7'",
        "version": "==3.1.2"
    },
    "mako": {
        "hashes": [
```

```
        "sha256:c97c79c018b9165ac9922ae4f32da095ffd3c4e6872b45eded42926deea46818",
        "sha256:d60a3903dc3bb01a18ad6a89cdbe2e4eadc69c0bc8ef1e3773ba53d44c3f7a34"
      ],
      "markers": "python_version >= '3.7'",
      "version": "==1.2.4"
    },
    "markupsafe": {
      "hashes": [
        "sha256:0212a68688482dc52b2d45013df70d169f542b7394fc744c02a57374a4207003",
        "sha256:089cf3dbf0cd6c100f02945abeb18484bd1ee57a079aefd52cffd17fba910b88",
        "sha256:10c1bfff05d95783da83491be968e8fe789263689c02724e0c691933c52994f5",
        "sha256:33b74d289bd2f5e527beadcaa3f401e0df0a89927c1559c8566c066fa4248ab7",
        "sha256:3799351e2336dc91ea70b034983ee71cf2f9533cdff7c14c90ea126bfd95d65a",
        "sha256:3ce11ee3f23f79dbd06fb3d63e2f6af7b12db1d46932fe7bd8afa259a5996603",
        "sha256:421be9fbf0ffe9ffd7a378aafebbf6f4602d564d34be190fc19a193232fd12b1",
        "sha256:43093fb83d8343aac0b1baa75516da6092f58f41200907ef92448ecab8825135",
        "sha256:46d00d6cfecdde84d40e572d63735ef81423ad31184100411e6e3388d405e247",
        "sha256:4a33dea2b688b3190ee12bd7cfa29d39c9ed176bda40bfa11099a3ce5d3a7ac6",
        "sha256:4b9fe39a2ccc108a4accc2676e77da025ce383c108593d65cc909add5c3bd601",
        "sha256:56442863ed2b06d19c37f94d999035e15ee982988920e12a5b4ba29b62ad1f77",
        "sha256:671cd1187ed5e62818414afe79ed29da836dde67166a9fac6d435873c44fdd02",
        "sha256:694deca8d702d5db21ec83983ce0bb4b26a578e71fbdbd4fdcd387daa90e4d5e",
        "sha256:6a074d34ee7a5ce3effbc526b7083ec9731bb3cbf921bbe1d3005d4d2bdb3a63",
        "sha256:6d0072fea50feec76a4c418096652f2c3238eaa014b2f94aeb1d56a66b41403f",
        "sha256:6fbf47b5d3728c6aea2abb0589b5d30459e369baa772e0f37a0320185e87c980",
        "sha256:7f91197cc9e48f989d12e4e6fbc46495c446636dfc81b9ccf50bb0ec74b91d4b",
        "sha256:86b1f75c4e7c2ac2ccdaec2b9022845dbb81880ca318bb7a0a01fbf7813e3812",
        "sha256:8dc1c72a69aa7e082593c4a203dcf94ddb74bb5c8a731e4e1eb68d031e8498ff",
        "sha256:8e3dcf21f367459434c18e71b2a9532d96547aef8a871872a5bd69a715c15f96",
        "sha256:8e576a51ad59e4bfaac456023a78f6b5e6e7651dcd383bcc3e18d06f9b55d6d1",
        "sha256:96e37a3dc86e80bf81758c152fe66dbf60ed5eca3d26305edf01892257049925",
        "sha256:97a68e6ada378df82bc9f16b800ab77cbf4b2fada0081794318520138c088e4a",
        "sha256:99a2a507ed3ac881b975a2976d59f38c19386d128e7a9a18b7df6fff1fd4c1d6",
        "sha256:a49907dd8420c5685cfa064a1335b6754b74541bbb3706c259c02ed65b644b3e",
        "sha256:b09bf97215625a311f669476f44b8b318b075847b49316d3e28c08e41a7a573f",
        "sha256:b7bd98b796e2b6553da7225aeb61f447f80a1ca64f41d83612e6139ca5213aa4",
        "sha256:b87db4360013327109564f0e591bd2a3b318547bcef31b468a92ee504d07ae4f",
        "sha256:bcb3ed405ed3222f9904899563d6fc492ff75cce56cba05e32eff40e6acbeaa3",
        "sha256:d4306c36ca495956b6d568d276ac11fdd9c30a36f1b6eb928070dc5360b22e1c",
        "sha256:d5ee4f386140395a2c818d149221149c54849dfcfcb9f1debfe07a8b8bd63f9a",
        "sha256:dda30ba7e87fbbb7eab1ec9f58678558fd9a6b8b853530e176eabd064da81417",
        "sha256:e04e26803c9c3851c931eac40c695602c6295b8d432cbe78609649ad9bd2da8a",
        "sha256:e1c0b87e09fa55a220f058d1d49d3fb8df88fbfab58558f1198e08c1e1de842a",
        "sha256:e72591e9ecd94d7feb70c1cbd7be7b3ebea3f548870aa91e2732960fa4d57a37",
        "sha256:e8c843bbcda3a2f1e3c2ab25913c80a3c5376cd00c6e8c4a86a89a28c8dc5452",
        "sha256:efc1913fd2ca4f334418481c7e595c00aad186563bbc1ec76067848c7ca0a933",
        "sha256:f121a1420d4e173a5d96e47e9a0c0dcff965afdf1626d28de1460815f7c4ee7a",
        "sha256:fc7b548b17d238737688817ab67deebb30e8073c95749d55538ed473130ec0c7"
      ],
      "markers": "python_version >= '3.7'",
      "version": "==2.1.1"
    },
    "marshmallow": {
      "hashes": [
        "sha256:90032c0fd650ce94b6ec6dc8dfeb0e3ff50c144586462c389b81a07205bedb78",
        "sha256:93f0958568da045b0021ec6aeb7ac37c81bfcccbb9a0e7ed8559885070b3a19b"
      ],
      "markers": "python_version >= '3.7'",
      "version": "==3.19.0"
    },
    "marshmallow-enum": {
      "hashes": [
        "sha256:38e697e11f45a8e64b4a1e664000897c659b60aa57bfa18d44e226a9920b6e58",
        "sha256:57161ab3dbfde4f57adeb12090f39592e992b9c86d206d02f6bd03ebec60f072"
      ],
      "index": "pypi",
      "version": "==1.5.1"
    },
```

```
    "marshmallow-sqlalchemy": {
      "hashes": [
        "sha256:aa376747296780a56355e3067b9c8bf43a2a1c44ff985de82b3a5d9e161ca2b8",
        "sha256:dbb061c19375eca3a7d18358d2ca8bbaee825fc3000a3f114e2698282362b536"
      ],
      "index": "pypi",
      "version": "==0.28.1"
    },
    "packaging": {
      "hashes": [
        "sha256:dd47c42927d89ab911e606518907cc2d3a1f38bbd026385970643f9c5b8ecfeb",
        "sha256:ef103e05f519cdc783ae24ea4e2e0f508a9c99b2d4969652eed6a2e1ea5bd522"
      ],
      "markers": "python_version >= '3.6'",
      "version": "==21.3"
    },
    "psycopg2": {
      "hashes": [
        "sha256:093e3894d2d3c592ab0945d9eba9d139c139664dcf83a1c440b8a7aa9bb21955",
        "sha256:190d51e8c1b25a47484e52a79638a8182451d6f6dff99f26ad9bd81e5359a0fa",
        "sha256:1a5c7d7d577e0eabfcf15eb87d1e19314c8c4f0e722a301f98e0e3a65e238b4e",
        "sha256:1e5a38aa85bd660c53947bd28aeaafb6a97d70423606f1ccb044a03a1203fe4a",
        "sha256:322fd5fca0b1113677089d4ebd5222c964b1760e361f151cbb2706c4912112c5",
        "sha256:4cb9936316d88bfab614666eb9e32995e794ed0f8f6b3b718666c22819c1d7ee",
        "sha256:920bf418000dd17669d2904472efeab2b20546efd0548139618f8fa305d1d7ad",
        "sha256:922cc5f0b98a5f2b1ff481f5551b95cd04580fd6f0c72d9b22e6c0145a4840e0",
        "sha256:a5246d2e683a972e2187a8714b5c2cf8156c064629f9a9b1a873c1730d9e245a",
        "sha256:b9ac1b0d8ecc49e05e4e182694f418d27f3aedcfca854ebd6c05bb1cffa10d6d",
        "sha256:d3ef67e630b0de0779c42912fe2cbae3805ebaba30cda27fea2a3de650a9414f",
        "sha256:f5b6320dbc3cf6cfb9f25308286f9f7ab464e65cfb105b64cc9c52831748ced2",
        "sha256:fc04dd5189b90d825509caa510f20d1d504761e78b8dfb95a0ede180f71d50e5"
      ],
      "index": "pypi",
      "version": "==2.9.5"
    },
    "pyjwt": {
      "hashes": [
        "sha256:69285c7e31fc44f68a1feb309e948e0df53259d579295e6cfe2b1792329f05fd",
        "sha256:d83c3d892a77bbb74d3e1a2cfa90afaadb60945205d1095d9221f04466f64c14"
      ],
      "markers": "python_version >= '3.7'",
      "version": "==2.6.0"
    },
    "pyparsing": {
      "hashes": [
        "sha256:2b020ecf7d21b687f219b71ecad3631f644a47f01403fa1d1036b0c6416d70fb",
        "sha256:5026bae9a10eeaefb61dab2f09052b9f4307d44aee4eda64b309723d8d206bbc"
      ],
      "markers": "python_full_version >= '3.6.8'",
      "version": "==3.0.9"
    },
    "pytz": {
      "hashes": [
        "sha256:222439474e9c98fced559f1709d89e6c9cbf8d79c794ff3eb9f8800064291427",
        "sha256:e89512406b793ca39f5971bc999cc538ce125c0e51c27941bef4568b460095e2"
      ],
      "version": "==2022.6"
    },
    "requests": {
      "hashes": [
        "sha256:7c5599b102feddaa661c826c56ab4fee28bfd17f5abca1ebbe3e7f19d7c97983",
        "sha256:8fefa2a1a1365bf5520aac41836fbee479da67864514bdb821f31ce07ce65349"
      ],
      "index": "pypi",
      "version": "==2.28.1"
    },
    "six": {
      "hashes": [
```

```
        "sha256:1e61c37477a1626458e36f7b1d82aa5c9b094fa4802892072e49de9c60c4c926",
        "sha256:8abb2f1d86890a2dfb989f9a77cfcfd3e47c2a354b01111771326f8aa26e0254"
      ],
      "markers": "python_version >= '2.7' and python_version not in '3.0, 3.1, 3.2, 3.3'",
      "version": "==1.16.0"
    },
    "sqlalchemy": {
      "hashes": [
        "sha256:0be9b479c5806cece01f1581726573a8d6515f8404e082c375b922c45cfc2a7b",
        "sha256:17aee7bfcef7bf0dea92f10e5dfdd67418dcf6fe0759f520e168b605855c003e",
        "sha256:21f3df74a0ab39e1255e94613556e33c1dc3b454059fe0b365ec3bbb9ed82e4a",
        "sha256:237067ba0ef45a518b64606e1807f7229969ad568288b110ed5f0ca714a3ed3a",
        "sha256:2dda5f96719ae89b3ec0f1b79698d86eb9aecb1d54e990abb3fdd92c04b46a90",
        "sha256:393f51a09778e8984d735b59a810731394308b4038acdb1635397c2865dae2b6",
        "sha256:3ca21b35b714ce36f4b8d1ee8d15f149db8eb43a472cf71600bf18dae32286e7",
        "sha256:3cbdbed8cdcae0f83640a9c44fa02b45a6c61e149c58d45a63c9581aba62850f",
        "sha256:3eba07f740488c3a125f17c092a81eeae24a6c7ec32ac9dbc52bf7afaf0c4f16",
        "sha256:3f68eab46649504eb95be36ca529aea16cd199f080726c28cbdbcbf23d20b2a2",
        "sha256:4c56e6899fa6e767e4be5d106941804a4201c5cb9620a409c0b80448ec70b656",
        "sha256:53f90a2374f60e703c94118d21533765412da8225ba98659de7dd7998641ab17",
        "sha256:595b185041a4dc5c685283ea98c2f67bbfa47bb28e4a4f5b27ebf40684e7a9f8",
        "sha256:65a0ad931944fcb0be12a8e0ac322dbd3ecf17c53f088bc10b6da8f0caac287b",
        "sha256:68e0cd5d32a32c4395168d42f2fefbb03b817ead3a8f3704b8bd5697c0b26c24",
        "sha256:6a06c2506c41926d2769f7968759995f2505e31c5b5a0821e43ca5a3ddb0e8ae",
        "sha256:6d7e1b28342b45f19e3dea7873a9479e4a57e15095a575afca902e517fb89652",
        "sha256:6f0ea4d7348feb5e5d0bf317aace92e28398fa9a6e38b7be9ec1f31aad4a8039",
        "sha256:7313e4acebb9ae88dbde14a8a177467a7625b7449306c03a3f9f309b30e163d0",
        "sha256:7cf7c7adbf4417e3f46fc5a2dbf8395a5a69698217337086888f79700a12e93a",
        "sha256:80ead36fb1d676cc019586ffdc21c7e906ce4bf243fe4021e4973dae332b6038",
        "sha256:9470633395e5f24d6741b4c8a6e905bce405a28cf417bba4ccbaadf3dab0111d",
        "sha256:94c0093678001f5d79f2dcbf3104c54d6c89e41ab50d619494c503a4d3f1aef2",
        "sha256:95f4f8d62589755b507218f2e3189475a4c1f5cc9db2aec772071a7dc6cd5726",
        "sha256:9c857676d810ca196be73c98eb839125d6fa849bfa3589be06201a6517f9961c",
        "sha256:a22208c1982f1fe2ae82e5e4c3d4a6f2445a7a0d65fb7983a3d7cbbe3983f5a4",
        "sha256:ad5f966623905ee33694680dda1b735544c99c7638f216045d21546d3d8c6f5b",
        "sha256:ae1ed1ebc407d2f66c6f0ec44ef7d56e3f455859df5494680e2cf89dad8e3ae0",
        "sha256:afd1ac99179d1864a68c06b31263a08ea25a49df94e272712eb2824ef151e294",
        "sha256:b6a337a2643a41476fb6262059b8740f4b9a2ec29bf00ffb18c18c080f6e0aed",
        "sha256:b737fbeb2f78926d1f59964feb287bbbd050e7904766f87c8ce5cfb86e6d840c",
        "sha256:c46322354c58d4dc039a2c982d28284330f8919f31206894281f4b595b9d8dbe",
        "sha256:c7e3b9e01fdbe1ce3a165cc7e1ff52b24813ee79c6df6dee0d1e13888a97817e",
        "sha256:c9aa372b295a36771cffc226b6517df3011a7d146ac22d19fa6a75f1cdf9d7e6",
        "sha256:d3b6d4588994da73567bb00af9d7224a16c8027865a8aab53ae9be83f9b7cbd1",
        "sha256:d3b9ac11f36ab9a726097fba7c7f6384f0129aedb017f1d4d1d4fce9052a1320",
        "sha256:d654870a66027af3a26df1372cf7f002e161c6768ebe4c9c6fdc0da331cb5173",
        "sha256:d8080bc51a775627865e0f1dbfc0040ff4ace685f187f6036837e1727ba2ed10",
        "sha256:da60b98b0f6f0df9fbf8b72d67d13b73aa8091923a48af79a951d4088530a239",
        "sha256:f5e8ed9cde48b76318ab989deeddc48f833d2a6a7b7c393c49b704f67dedf01d",
        "sha256:f8e5443295b218b08bef8eb85d31b214d184b3690d99a33b7bd8e5591e2b0aa1"
      ],
      "index": "pypi",
      "version": "==1.4.44"
    },
    "urllib3": {
      "hashes": [
        "sha256:3fa96cf423e6987997fc326ae8df396db2a8b7c667747d47ddd8ecba91f4a74e",
        "sha256:b930dd878d5a8afb066a637fbb35144fe7901e3b209d1cd4f524bd0e9deee997"
      ],
      "markers": "python_version >= '2.7' and python_version not in '3.0, 3.1, 3.2, 3.3, 3.4, 3.5' and python_version < '4'",
      "version": "==1.26.12"
    },
    "waitress": {
      "hashes": [
        "sha256:7500c9625927c8ec60f54377d590f67b30c8e70ef4b8894214ac6e4cad233d2a",
        "sha256:780a4082c5fbc0fde6a2fcfe5e26e6efc1e8f425730863c04085769781f51eba"
      ],
      "index": "pypi",
      "version": "==2.1.2"
```

```
        },
        "werkzeug": {
            "hashes": [
                "sha256:7ea2d48322cc7c0f8b3a215ed73eabd7b5d75d0b50e31ab006286ccff9e00b8f",
                "sha256:f979ab81f58d7318e064e99c4506445d60135ac5cd2e177a2de0089bfd4c9bd5"
            ],
            "markers": "python_version >= '3.7'",
            "version": "==2.2.2"
        },
        "zipp": {
            "hashes": [
                "sha256:4fcb6f278987a6605757302a6e40e896257570d11c51628968ccb2a47e80c6c1",
                "sha256:7a7262fd930bd3e36c50b9a64897aec3fafff3dfdeec9623ae22b40e93f99bb8"
            ],
            "markers": "python_version >= '3.7'",
            "version": "==3.10.0"
        }
    },
    "develop": {}
}
```

problemapi.py

```
from flask import request
from flask_restful import Resource
from alembic.database import db_session
from alembic.models import Service_Center, User, Problem, Device, branchEnum
from schemas import ProblemSchema
from marshmallow.exceptions import ValidationError
from custom import json_error, errs
from main import bcrypt, master_required, admin_required, get_user
from flask_jwt_extended import jwt_required
from flask_jwt_extended import create_access_token

problem_schema = ProblemSchema()

class ProblemsAPI(Resource):
    @jwt_required()
    def get(self):
        problems = Problem.query.all()
        return problem_schema.dump(problems, many=True), 200

    @master_required()
    def post(self):
        json_data = request.get_json()
        if not json_data:
            return errs.bad_request
        problem = Problem.query.get(json_data.get('problemid', None))
        if problem:
            return errs.exists
        try:
            data = problem_schema.load(json_data, session=db_session)
        except ValidationError as err:
            return json_error(err.messages, 400)
        db_session.add(data)
        db_session.commit()
        return {"message": "Created problem.", "problem": json_data}

    @admin_required()
    def put(self):
        json_data = request.get_json()
        if not json_data:
            return errs.bad_request
        problem = Problem.query.get(json_data.get('problemid', None))
        if not problem:
```

```
            return errs.not_found
        try:
            data = problem_schema.load(json_data, session=db_session)
        except ValidationError as err:
            return json_error(err.messages, 400)
        db_session.add(data)
        db_session.commit()

        return {"message": "Updated problem.", "problem": json_data}


class ProblemAPI(Resource):
    @jwt_required()
    def get(self, problemid):
        problem = Problem.query.get(problemid)
        if not problem:
            return errs.not_found
        return problem_schema.dump(problem), 200

    @admin_required()
    def delete(self, problemid):
        problem = Problem.query.get(problemid)
        if not problem:
            return errs.not_found
        db_session.delete(problem)
        db_session.commit()
        return '', 204


class ProblemBranchAPI(Resource):
    @jwt_required()
    def get(self):
        json_data = request.get_json()
        if not json_data:
            return errs.bad_request
        branch = json_data.get('branch', None)
        branchEnums = [e.name for e in branchEnum]
        if branch not in branchEnums:
            return json_error('Invalid request. Bad status value. Must be on of: ' + ', '.join(branchEnums), 400)
        problems = Problem.query.filter_by(branch=branch)
        return problem_schema.dump(problems, many=True), 200
```

schemas.py

```
from alembic.models import Service_Center, User, Problem, Device, deviceStatus, branchEnum, userStatus
from marshmallow_sqlalchemy import SQLAlchemyAutoSchema, auto_field
from marshmallow_enum import EnumField


class ServiceCenterSchema(SQLAlchemyAutoSchema):
    class Meta:
        model = Service_Center
        # include_relationships = True
        load_instance = True
        include_fk = True

class UserSchema(SQLAlchemyAutoSchema):
    userStatus = EnumField(userStatus)
    class Meta:
        model = User
        # include_relationships = True
        load_instance = True
        include_fk = True
```

```python
class ProblemSchema(SQLAlchemyAutoSchema):
    branch = EnumField(branchEnum)
    class Meta:
        model = Problem
        # include_relationships = True
        load_instance = True
        include_fk = True

class DeviceSchema(SQLAlchemyAutoSchema):
    status = EnumField(deviceStatus)

    class Meta:
        model = Device
        # include_relationships = True
        load_instance = True
        include_fk = True
```

serviceapi.py

```python
from flask import request
from flask_restful import Resource
from alembic.database import db_session
from alembic.models import Service_Center, User, Problem, Device, deviceStatus
from schemas import ServiceCenterSchema
from marshmallow.exceptions import ValidationError
from custom import json_error, errs
from main import bcrypt, master_required, admin_required, get_user
from flask_jwt_extended import jwt_required
from flask_jwt_extended import create_access_token

service_schema = ServiceCenterSchema()


class ServicesAPI(Resource):

    def get(self):
        services = Service_Center.query.all()
        return service_schema.dump(services, many=True), 200

    @admin_required()
    def post(self):
        json_data = request.get_json()
        if not json_data:
            return errs.bad_request
        service = Service_Center.query.get(json_data.get('serviceid', None))
        if service:
            return errs.exists
        try:
            data = service_schema.load(json_data, session=db_session)
        except ValidationError as err:
            return json_error(err.messages, 400)
        db_session.add(data)
        db_session.commit()

        return {"message": "Created service.", "service": json_data}

    @master_required()
    def put(self):
        json_data = request.get_json()
        if not json_data:
            return errs.bad_request
```

```
        service = Service_Center.query.get(json_data.get('serviceid', None))
        if not service:
            return errs.not_found
        claims = get_user()
        if claims.role != 'admin' and claims.service_centerid != json_data.get(
                'serviceid', None):
            return json_error("Forbidden. Only master can update info on their service.", 403)
        try:
            data = service_schema.load(json_data, session=db_session)
        except ValidationError as err:
            return json_error(err.messages, 400)
        db_session.add(data)
        db_session.commit()

        return {"message": "Updated service.", "service": json_data}


class ServiceAPI(Resource):
    def get(self, serviceid):
        service = Service_Center.query.get(serviceid)
        if not service:
            return errs.not_found
        return service_schema.dump(service), 200

    @admin_required()
    def delete(self, serviceid):
        service = Service_Center.query.get(serviceid)
        if not service:
            return errs.not_found
        db_session.delete(service)
        db_session.commit()
        return '', 204


class ServiceDeviceAPI(Resource):
    @admin_required()
    def get(self):
        statuses = [e.name for e in deviceStatus] + [None]
        result = {}

        for status in statuses:
            result[status] = len(Device.query.filter_by(status=status).all())
        return result, 200
```

userapi.py

```
from flask import request, jsonify
from flask_restful import Resource
from alembic.database import db_session
from alembic.models import User, User, Problem, Device
from schemas import UserSchema, DeviceSchema
from marshmallow.exceptions import ValidationError
from custom import json_error, errs
from main import bcrypt, master_required, admin_required, get_user
from flask_jwt_extended import jwt_required, verify_jwt_in_request
from flask_jwt_extended import create_access_token

user_schema = UserSchema()
device_schema = DeviceSchema()


class UserLoginAPI(Resource):
    def post(self):
```

```
        username = request.json.get("username", None)
        password = request.json.get("password", None)
        user = User.query.filter_by(username=username)[0]
        if not user:
            return json_error("User not found", 404)
        if user.password != password:
            return json_error("Invalid password", 403)
        access_token = create_access_token(identity=username)
        return jsonify(access_token=access_token)


class UsersAPI(Resource):
    @admin_required()
    def get(self):
        users = User.query.all()
        users_data = user_schema.dump(users, many=True)
        for data in users_data:
            data['password'] = bcrypt.generate_password_hash(data['password']).decode('utf-8')
        return users_data, 200

    def post(self):
        json_data = request.get_json()
        if not json_data:
            return errs.bad_request
        user = User.query.get(json_data.get('userid', None))
        if user:
            return errs.exists
        try:
            data = user_schema.load(json_data, session=db_session)
        except ValidationError as err:
            return json_error(err.messages, 400)
        roles = ['admin', 'master', 'client', None]
        if data.role not in roles:
            return json_error(f'Bad request. Invalid role. Must be one of these: {", ".join(roles[:-1])}', 400)
        if data.role == 'admin' or data.role == 'master':
            verify_jwt_in_request()
            # claims = get_user()
            # if claims.role != 'admin':
            #     return json_error("Forbidden. Only admins can assign roles", 403)
        db_session.add(data)
        db_session.commit()
        json_data['password'] = bcrypt.generate_password_hash(json_data['password']).decode('utf-8')
        return {"message": "Created user.", "user": json_data}

    @jwt_required()
    def put(self):
        json_data = request.get_json()
        if not json_data:
            return errs.bad_request
        user = User.query.get(json_data.get('userid', None))
        claims = get_user()
        role = claims.role
        if not user:
            return errs.not_found
        if claims.role != 'admin' and claims.userid != json_data.get('userid', None):
            return json_error("Forbidden. Clients can only update themselves.", 403)
        try:
            data = user_schema.load(json_data, session=db_session)
        except ValidationError as err:
            return json_error(err.messages, 400)
        roles = ['admin', 'master', 'client', None]
        if data.role not in roles:
            return json_error(f'Bad request. Invalid role. Must be one of these: {", ".join(roles[:-1])}', 400)
        if data.role != None:
            if role != 'admin' and role != data.role:
                return json_error("Forbidden. Only admins can assign roles", 403)
        db_session.add(data)
        db_session.commit()
```

```
        return {"message": "Updated user.", "user": json_data}


class UserAPI(Resource):
    @jwt_required()
    def get(self, username):
        user = User.query.filter_by(username=username).first()
        if not user:
            return errs.not_found
        claims = get_user()
        if claims.role != 'admin' and claims.role != 'master' and claims.username != username:
            return json_error("Forbidden. Client can only get info about himself.", 403)
        data = user_schema.dump(user)
        data['password'] = bcrypt.generate_password_hash(data['password']).decode('utf-8')
        return data, 200

    @jwt_required()
    def delete(self, username):
        user = User.query.filter_by(username=username).first()
        if not user:
            return errs.not_found
        claims = get_user()
        if claims.role != 'admin' and claims.username != username:
            return json_error("Forbidden. Client can only delete himself.", 403)
        db_session.delete(user)
        db_session.commit()
        return '', 204


class UserDeviceAPI(Resource):
    @jwt_required()
    def get(self, username):
        user = User.query.filter_by(username=username).first()
        if not user:
            return errs.not_found
        claims = get_user()
        if claims.role != 'admin' and claims.role != 'master' and claims.username != username:
            return json_error("Forbidden. Clients can only get info of their own devices.", 403)
        if claims.role == 'master' and user.service_centerid != claims.service_centerid:
            return json_error("Forbidden. Master can only get info of his clients.", 403)
        data = user_schema.dump(user)
        data['password'] = bcrypt.generate_password_hash(data['password']).decode('utf-8')
        return {'user': data, 'devices': device_schema.dump(user.device, many=True)}, 200
```

tests.py

```
from schemas import *
from main import app, bcrypt
from werkzeug.security import generate_password_hash, check_password_hash

app.testing = True
client = app.test_client()


def check_data(responce_json, compare_object):
    for x in compare_object.keys():
        if x == 'password':
            continue
        else:
            assert responce_json[x] == compare_object[x]


global test_service_id, test_service_json
test_service_id = 'not found'
test_service_json = {
    'serviceid':1,
```

```
    'city':'Lviv',
    'adress':'Bandery',
    'phone':'+1234567890',
    'rating':'10'
}

global admin_token
admin_token = ''

def test_login_admin():
    global admin_token
    res = client.post(f'/api/v1/login' ,json={
        "username": 'admin',
        "password": 'admin'
    })
    assert res.status_code == 200
    admin_token=res.json["access_token"]

global test_user_id
test_user_id = 'not found'
test_user_json = {
    'userid':1,
    'username':'usern',
    'firstname': 'John2',
    'lastname': 'Jam2es2',
    'email': 'joh2n.james.2022@email.com',
    'password': 'qwerty22022',
    'userStatus':"active",
    'service_centerid' : 1,
    "role":'master'
}
def authentication_headers(username = test_user_json['username']):
    return {'headers': {
            'Authorization': f'"Bearer {username}"'
        }
    }
def test_create_service():
    global test_service_id, test_service_json, admin_token
    res = client.post('/api/v1/service', json=test_service_json, **authentication_headers(admin_token))
    assert res.status_code == 200
    check_data(res.json['service'], test_service_json)
    # assert res.json.get('serviceid')
    test_service_json = res.json["service"]

def test_create_service2():
    global test_service_id, test_service_json, admin_token
    res = client.post('/api/v1/service', json=test_service_json, **authentication_headers(admin_token))
    assert res.status_code == 403


def test_create_user():
    global test_user_json, admin_token
    res = client.post(f'/api/v1/user', json=test_user_json, **authentication_headers(admin_token))
    assert res.status_code == 200
    check_data(res.json['user'], test_user_json)

def test_create_user2():
    global test_user_json, admin_token
    res = client.post(f'/api/v1/user', json=test_user_json, **authentication_headers(admin_token))
    assert res.status_code == 403

global test_token
test_token = ''

def test_login_user():
    global test_user_json, test_token
    res = client.post(f'/api/v1/login' ,json={
        "username": test_user_json["username"],
```

```
        "password": test_user_json["password"]
    })
    assert res.status_code == 200
    test_token=res.json["access_token"]

def test_get_user():
    global admin_token
    res = client.get(f'/api/v1/user', **authentication_headers(admin_token))
    assert res.status_code == 200

def test_get_service():
    global admin_token
    res = client.get(f'/api/v1/service', **authentication_headers(admin_token))
    assert res.status_code == 200

def test_get_user_username():
    global test_user_json, test_token
    res = client.get(f'/api/v1/user/{test_user_json["username"]}', **authentication_headers(admin_token))
    assert res.status_code == 200

def test_update_user():
    global test_user_json
    test_user_json['firstname'] = 'John3'
    res = client.put('/api/v1/user', json=test_user_json, **authentication_headers(test_token))
    assert res.status_code == 200
    check_data(res.json['user'], test_user_json)
    test_user_json['firstname'] = 'John2'
    res = client.put('/user', json=test_user_json, **authentication_headers(test_token))

def test_update_service():
    global test_service_json, admin_token
    test_user_json['adress'] = 'Chuprinki'
    res = client.put('/api/v1/service', json=test_service_json, **authentication_headers(admin_token))
    assert res.status_code == 200
    test_user_json['adress'] = 'Bandery'
    res = client.put('/service', json=test_service_json, **authentication_headers(admin_token))


def test_get_service_id():
    global test_service_id, admin_token
    res = client.get(f'/api/v1/service/{test_service_json["serviceid"]}', **authentication_headers(admin_token))
    assert res.status_code == 200

def test_get_service_id1():
    global test_service_id, admin_token
    res = client.get(f'/api/v1/service/{test_service_json["serviceid"]+1}', **authentication_headers(admin_token))
    assert res.status_code == 404


global test_problem_id, test_problem_json
test_problem_id = 'not found'
test_problem_json = {
    'problemid':1,
    'branch':'screen',
    'daysToSolve':3
}

def test_create_problem():
    global test_problem_json, admin_token
    res = client.post('/api/v1/problem', json=test_problem_json, **authentication_headers(admin_token))
    assert res.status_code == 200

def test_create_proble3():
    global test_problem_json, admin_token
    res = client.post('/api/v1/problem', json=test_problem_json, **authentication_headers(admin_token))
    assert res.status_code == 403

def test_get_problem():
```

```
    global admin_token
    res = client.get(f'/api/v1/problem', **authentication_headers(admin_token))
    assert res.status_code == 200

def test_update_problem():
    global test_problem_json, admin_token
    test_problem_json['branch'] = 'button'
    res = client.put('/api/v1/problem', json=test_problem_json, **authentication_headers(admin_token))
    assert res.status_code == 200
    test_problem_json['branch'] = 'screen'
    res = client.put('/problem', json=test_problem_json, **authentication_headers(admin_token))

def test_get_problem_id():
    global test_problem_json, admin_token
    res = client.get(f'/api/v1/problem/{test_problem_json["problemid"]}', **authentication_headers(admin_token))
    assert res.status_code == 200

def test_get_problem_id1():
    global test_problem_json, admin_token
    res = client.get(f'/api/v1/problem/{test_problem_json["problemid"]+1}', **authentication_headers(admin_token))
    assert res.status_code == 404

global test_device_id, test_device_json
test_device_id = 'not found'
test_device_json = {
    'deviceid':1,
    'firm':'Samsung',
    'model':'S10',
    'status':'arrived',
    'userid':1
}

def test_create_device():
    global test_device_json, test_token
    res = client.post('/api/v1/device', json=test_device_json, **authentication_headers(test_token))
    assert res.status_code == 200

def test_create_device2():
    global test_device_json, test_token
    res = client.post('/api/v1/device', json=test_device_json, **authentication_headers(test_token))
    assert res.status_code == 403

def test_get_device():
    global admin_token
    res = client.get(f'/api/v1/device', **authentication_headers(admin_token))
    assert res.status_code == 200

def test_update_device():
    global test_device_json, admin_token
    test_device_json['firm'] = 'LG'
    res = client.put('/api/v1/device', json=test_device_json, **authentication_headers(admin_token))
    assert res.status_code == 200
    test_device_json['firm'] = 'Samsung'
    res = client.put('/device', json=test_device_json, **authentication_headers(admin_token))

def test_get_device_status():
    global test_device_json, admin_token
    res = client.get(f'/api/v1/device/findByStatus', json = { "status":"arrived" },
                     **authentication_headers(admin_token))
    assert res.status_code == 200

def test_get_device_status1():
    global test_device_json, admin_token
    res = client.get(f'/api/v1/device/findByStatus', json = { "status":"arrrived" },
                     **authentication_headers(admin_token))
    assert res.status_code == 400

def test_get_device_firm():
```

```
    global test_device_json, admin_token
    res = client.get(f'/api/v1/device/findByFirm', json = { "firm":"Samsung" },
                     **authentication_headers(admin_token))
    assert res.status_code == 200



def test_get_device_id():
    global test_device_json, admin_token
    res = client.get(f'/api/v1/device/{test_device_json["deviceid"]}', **authentication_headers(admin_token))
    assert res.status_code == 200

def test_get_device_problems():
    global test_device_json, admin_token
    res = client.get(f'/api/v1/device/{test_device_json["deviceid"]}/problem', **authentication_headers(admin_token))
    assert res.status_code == 200

def test_get_user_devices():
    global test_user_json, test_token
    res = client.get(f'/api/v1/user/{test_user_json["username"]}/device', **authentication_headers(admin_token))
    assert res.status_code == 200

def test_get_service_device():
    global admin_token
    res = client.get(f'/api/v1/service/device', **authentication_headers(admin_token))
    assert res.status_code == 200

def test_get_problem_branch():
    global test_problem_json, admin_token
    res = client.get(f'/api/v1/problem/findByBranch', json = { "branch":"screen" },
                     **authentication_headers(admin_token))
    assert res.status_code == 200

def test_get_problem_branch1():
    global test_problem_json, admin_token
    res = client.get(f'/api/v1/problem/findByBranch', json = { "branch":"screeen" },
                     **authentication_headers(admin_token))
    assert res.status_code == 400

def test_add_problem_device():
    global test_device_json, test_problem_json, admin_token
    res = client.post(f'/api/v1/device/{test_device_json["deviceid"]}/problem', json = test_problem_json,
                      **authentication_headers(admin_token))
    assert res.status_code == 200

def test_get_device_id_problem_id():
    global test_device_json, test_problem_json, admin_token
    res = client.get(f'/api/v1/device/{test_device_json["deviceid"]}/problem/{test_problem_json["problemid"]}',
                     **authentication_headers(admin_token))
    assert res.status_code == 200

def test_delete_problem_device():
    global test_device_json, test_problem_json, admin_token
    res = client.delete(f'/api/v1/device/{test_device_json["deviceid"]}/problem/{test_problem_json["problemid"]}',
                    **authentication_headers(admin_token))
    assert res.status_code == 204

def test_create_problem2():
    global test_problem_json, admin_token
    res = client.post('/api/v1/problem', json=test_problem_json, **authentication_headers(admin_token))
    assert res.status_code == 200

def test_update_device_problem():
    global test_service_json, test_problem_json, admin_token
    test_problem_json['daysToSolve'] = 1
    res = client.put(f'/api/v1/device/{test_device_json["deviceid"]}/problem/{test_problem_json["problemid"]}',
                     json=test_problem_json, **authentication_headers(admin_token))
    assert res.status_code == 200
```

```
    test_problem_json['daysToSolve'] = 3
    res = client.put('/service', json=test_service_json, **authentication_headers(admin_token))

def test_del_device1():
    global test_device_json,admin_token
    res = client.delete(f'/api/v1/device/{test_device_json["deviceid"]+1}', **authentication_headers(admin_token))
    assert res.status_code == 404

def test_del_device():
    global test_device_json,admin_token
    res = client.delete(f'/api/v1/device/{test_device_json["deviceid"]}', **authentication_headers(admin_token))
    assert res.status_code == 204

def test_del_service1():
    global test_service_id,test_token
    res = client.delete(f'/api/v1/service/{test_service_json["serviceid"]+1}', **authentication_headers(admin_token))
    assert res.status_code == 404

def test_del_service():
    global test_service_id,test_token
    res = client.delete(f'/api/v1/service/{test_service_json["serviceid"]}', **authentication_headers(admin_token))
    assert res.status_code == 204

def test_del_user1():
    global test_user_json, admin_token
    res = client.delete(f'/api/v1/user/Antoha', **authentication_headers(admin_token))
    assert res.status_code == 404

def test_del_user():
    global test_user_json, admin_token
    res = client.delete(f'/api/v1/user/{test_user_json["username"]}', **authentication_headers(admin_token))
    assert res.status_code == 204

def test_del_problem1():
    global test_problem_json,test_token
    res = client.delete(f'/api/v1/problem/{test_problem_json["problemid"]+1}', **authentication_headers(admin_token))
    assert res.status_code == 404

def test_del_problem():
    global test_problem_json,test_token
    res = client.delete(f'/api/v1/problem/{test_problem_json["problemid"]}', **authentication_headers(admin_token))
    assert res.status_code == 204
```